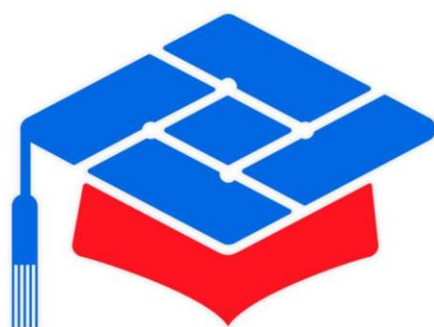




第十届

全国大学生集成电路创新创业大赛

报告类型： 总技术文档（设计、仿真文档）

参赛杯赛： 紫光同创杯

作品名称： 基于盘古50K+ARM的检测方案及实现

队伍编号： CICC1002661

团队名称： 请输入文本

目录

重要内容预览	3
一、关键技术	错误!未定义书签。
二、核心指标	错误!未定义书签。
三、作品亮点	错误!未定义书签。
1. 作品概述	4
1.1 题目背景与应用场景	6
1.2 项目简介	6
1.3 基本参数	7
1.4 设计指标	11
2. 系统总体设计	11
2.1 系统硬件架构	11
2.2 FPGA-ARM异构分工设计	13
2.3 通信方案选型	14
2.4 数据流与时钟域设计	15
2.5 模块与接口概述	16
3. FPGA端硬件设计	19
3.1 FPGA工程结构与模块划分	19
3.2 多源视频采集模块	20
3.2.1 OV5640摄像头初始化	20
3.2.2 8bit到16bit像素格式转换	22
3.2.3 视频源选择	22
3.3 实时锐化增强算法	23
3.3.1 设计动机	23
3.3.2 算法原理	23
3.3.3 三级流水线硬件实现	23
3.3.4 无符号数设计的鲁棒性	25
3.3.5 资源开销	25
3.4 DDR3帧缓存与读写控制	26
3.4.1 整体架构	26
3.4.2 写通路设计	27
3.4.3 读通路设计	27
3.4.4 跨时钟域处理	28
3.5 千兆以太网传输模块	28
3.5.1 GMII接口与RGMII桥接	29
3.5.2 异步FIFO跨时钟域设计	30
3.5.3 发送控制状态机	30
3.5.4 自定义图像封包格式	31
3.5.5 UDP/IP/MAC三层协议封装	32
3.5.6 帧率控制：DUMP_FIFO机制	33
3.6 HDMI显示输出模块	34

3.6.1	VGA/HDMI时序生成	34
3.6.2	HDMI编码器配置与像素格式转换	35
4.	ARM端软件设计	35
4.1	系统总体流程	35
4.2	YOLOv8车牌检测	36
4.3	PPOCRv4字符识别	40
4.4	中文标注显示	41
4.5	FPGA-ARM数据交互	41
5.	仿真与验证	42
5.1	行为级仿真: Feature Test	42
5.1.1	锐化模块 RTL 级时序与功能验证	42
5.1.2	视频时序发生器 (sync_vg) 功能仿真验证	43
5.1.3	UDP 传输控制与降频丢帧逻辑仿真验证	44
5.2	硬件验证	46
5.2.1	功耗参数	46
5.2.2	时序参数	47
5.2.3	时序检查	48
5.2.4	资源占用	49
6.	关于车牌识别的重要图片	51
7.	AI工具使用声明	55
(a)	使用的AI工具名称	55
(b)	使用场景与用途	55
(c)	AI生成内容占比	55

重要内容预览

作品名称：基于紫光同创FPGA的智能交通感知系统

本作品面向城市交通边缘计算场景，构建了一套基于紫光同创盘古50KFPGA与瑞芯微RK3588 ARM的异构实时感知平台。系统充分发挥FPGA的并行流水线优势完成图像增强与高速传输，利用RK3588内置6TOPS NPU完成AI推理，通过千兆以太网UDP协议实现二者高效数据交互，形成从视频采集、实时增强、网络传输到车牌识别与结果输出的全流程闭环。

一、关键技术

1. 自研1D极简锐化算法

基于拉普拉斯算子变体，采用三级流水线硬件架构，在84MHz像素时钟下实现固定3周期延迟的边缘对比度增强。算法完全采用无符号数设计，从架构层面规避了Verilog综合器中\$signed()的兼容性隐患。该模块不消耗任何DSP和BRAM资源，仅占用约200个LUT和150个FF，在FPGA整体LUT占用率17.39%中占比不足3%。

2. 完整UDP/IP硬件协议栈

以纯Verilog实现UDP/IP/MAC三层协议封装，支持自定义图像封包格式（8字节帧头+1280字节像素数据）。协议栈通过RGMII接口驱动RTL8211E千兆以太网PHY芯片，在125MHz时钟下实现线速发送。

3. 帧率分层控制：DUMP_FIFO黑洞机制

针对摄像头30fps物理输出与RK3588 NPU处理能力之间的带宽矛盾，提出DUMP_FIFO策略。维护5帧循环计数器（skip_cnt 0→4），仅skip_cnt=0时正常发送完整UDP包，skip_cnt=1-4时从异步FIFO中读出数据后直接丢弃、不唤醒网卡。该方案经历了三次迭代，最终以极简硬件修改实现了工业级流量控制效果，使ARM端接收帧率稳定在6fps。

4. YOLOv8+PP0CRv4双阶段AI识别

ARM端部署YOLOv8（车牌检测）与PP0CRv4（字符识别）双模型。YOLOv8经INT8量化后部署于RK3588 NPU，mAP@0.5达0.983；PP0CRv4配置覆盖GA 36-2018标准的中文字符集字典，支持蓝牌、绿牌、黄牌、军警、新能源等多种常见及特殊车牌。两个模型功能解耦，可独立优化和更新。针对双层车牌的布局特点，采用上下拼接预处理策略，无需单独训练即可兼容。

二、核心指标

指标类别	参数项	实测值
FPGA资源	LUT 占用	7,445 / 42,800 (17.39%)

指标类别	参数项	实测值
FPGA功耗	寄存器占用	7,276 / 64,200 (11.33%)
	BRAM占用	22.5 / 134 (16.79%)
	DSP占用	0 / 84 (0%)
	总功耗	1.182W
AI模型精度	静态功耗	0.219W
	结温	37.565° C
	YOLOv8 mAP@0.5	0.983 (单行车牌0.993 / 双行车牌0.973)
	训练/验证规模	15,000张 / 9,000张
系统性能	综合识别准确率	≥98%
	端到端系统延迟	约200ms
传输性能	FPGA输出帧率	6fps (可调)
	有效网络带宽	约80Mbps
支持车牌	常见类型	蓝牌、绿牌
	特殊类型	黄牌、军警、挂车、领馆、使馆、新能源等

三、作品亮点

架构层面：FPGA处理确定性像素级任务（采集、增强、传输），ARM/NPU处理语义级任务（检测、识别），二者通过千兆以太网耦合。同一套硬件架构无需任何修改即可扩展行人闯红灯检测、横穿马路预警等多种交通行为分析功能。

算法层面：锐化算法以零DSP/BRAM的极低成本实现图像质量有效提升，无符号数设计确保跨综合器的完全兼容性。双阶段AI方案在功能上解耦、在性能上互补，字符集通过字典文件灵活扩展，天然支持多类型车牌。综合识别准确率达到98%以上，系统端到端延迟约200ms，满足实时交通感知场景的工程需求。

工程层面：完整记录了降帧方案的三次迭代过程。FPGA端全部采用纯Verilog实现，不依赖闭源IP核，确保跨平台可移植性。系统已在MES50HP开发板上完成全流程验证，FPGA资源占用率低于20%，功耗仅1.182W，满足嵌入式边缘部署的低功耗要求。

1. 作品概述

1.1 题目背景与应用场景

随着城市的迅速发展，城市交通环境日趋复杂。行人行为及车牌识别作为智能交通的核心感知技术，在车辆通行管理、交通违章抓拍、ETC闸机、停车场自动化运维等场景中具有不可替代的作用。

传统基于CPU/GPU的行人和车牌识别方案存在实时性不足、功耗较高、部署灵活性受限等问题。而FPGA凭借并行计算架构、低延迟、高性价比的优势，配合ARM处理器强大的AI推理能力，成为嵌入式场景下智能交通感知系统的理想硬件平台。

本赛题要求基于紫光同创FPGA芯片，采用纯逻辑方案或FPGA+ARM异构方案，完成包含图像采集接口、预处理模块、车牌识别、结果输出的完整硬件架构，实现对蓝牌、绿牌及特殊车牌（军警、新能源等）的准确识别，我们队伍采用ARM+FPGA方案。

1.2 项目简介

本系统——“基于紫光同创FPGA的智能交通感知系统”——是一款面向城市交通边缘计算场景的FPGA+ARM异构实时感知平台。系统以紫光同创PGL50H FPGA为核心，承担图像采集、实时增强与高速传输等确定性像素级任务；以瑞芯微RK3588 ARM处理器为AI推理引擎，利用其内置6TOPS算力NPU，完成车牌检测与字符识别等语义级任务。二者通过千兆以太网UDP协议实现高速数据交互，形成从视频输入到识别结果输出的全流程闭环。

系统核心功能包括：

1. 多源视频接入：支持多种图像采集方式，视频源可通过顶层宏定义切换。

- 2. FPGA实时图像增强:应用1D锐化算法，对比度增强，输入图像质量高，确保了更好的鲁棒性。
- 3. 千兆以太网实时传输：基于Verilog实现UDP/IP/MAC三层硬件协议栈，通过RGMII接口驱动RTL8211E PHY芯片，以自定义封包格式实时发送。
- 4. AI车牌检测与识别：YOLOv8（主要工作为车牌定位）与PP0CRv3（字符识别）双阶段模型支持蓝牌、绿牌、双层车牌、摩托车牌、使馆领事馆、军警等多种车牌类型在强光、阴暗、正常光线环境下的高精度识别。

在实现基础功能的同时，系统设计亮点包括但不限于：

- 1. 异构协同架构：FPGA处理确定性像素级任务，ARM/NPU处理语义级任务，二者通过千兆以太网松耦合
- 2. 硬件锐化算法创新设计：通过纯无符号数设计消除综合器隐患，三级流水线实现单周期1像素吞吐，零DSP/BRAM占用，以极低硬件成本换取图像质量的有效提升。
- 3. 完整UDP硬件协议栈：相较于调用厂商IP核的方案，可移植性更高，资源开销更低。
- 4. 高可扩展性与便携性：同一套硬件架构仅需在ARM端增部署模型即可扩展更多功能。

1.3 基本参数

系统关键硬件参数与设计指标汇总

FPGA芯片

参数项	规格
型号	紫光同创 PGL50H-6FBG484
系列	Logos系列
工艺	40nm
等效LUT4资源	42,800
寄存器资源	64,200
DRM18K资源	134

参数项	规格
用户IO资源	296
PLL资源	4
HSST高速收发器	4路，最高6.375Gbps
DDR3最高速率	1066Mbps（16bit位宽）

ARM处理器

参数项	规格
型号	瑞芯微 RK3588
CPU	四核Cortex-A76 + 四核Cortex-A55
NPU	三核，6TOPS（INT8）
内存	8GB LPDDR4×2

图像采集

参数项	规格
摄像头型号	OV5640 × 2
分辨率	1280 × 720
帧率	30fps
像素格式	RGB565（16bit/像素）
像素时钟	84MHz

数据传输

参数项	规格
接口	千兆以太网RGMII
PHY芯片	Realtek RTL8211E
协议	UDP/IP
FPGA端IP	192.168.1.110:8080
ARM端IP	192.168.1.105:8080
RGMII时钟	125MHz
输出帧率	6fps（可调）
有效载荷/包	1288字节（8头+1280像素）

存储与显示

参数项	规格
DDR3	256MB，16bit位宽
HDMI输出	1280×720@60Hz，像素时钟67.5MHz

AI模型

参数项	规格
检测模型	YOLOv8（INT8量化，NPU部署）
识别模型	PPOCRv3
训练集	15,000张
验证集	9,000张
部署工具链	RKNN-Toolkit2

系统实测指标

参数项	实测值
FPGA LUT占用	7,445 / 42,800（17.39%）
FPGA 寄存器占用	7,276 / 64,200（11.33%）
FPGA BRAM占用	22.5 / 134（16.79%）
FPGA IO占用	138 / 296（46.62%）
FPGA DSP占用	0 / 84（0%）
FPGA 总功耗	1.182W（静态0.219W）
FPGA 结温	37.565℃
FPGA 温升裕量	62.435℃
端到端延迟（ARM显示）	< 100ms

FPGA时钟域

时钟信号	频率	用途
sys_clk	50MHz	外部晶振输入
ddrphy_clkin	100MHz	DDR3 PHY参考
pix_clk	67.5MHz	HDMI显示像素时钟
cfg_clk	10MHz	I2C/HDMI配置
clk_25M	25MHz	摄像头I2C
rgmii_rxc	125MHz（外部）	RGMII接收时钟

1.4 设计指标

1. 采集-识别-输出全链路：FPGA通过OV5640采集720P@30fps视频，经自研锐化增强后以UDP协议发送至RK3588，ARM端通过YOLOv8+PPOCRv3完成识别，结果输出至HDMI显示器，形成完整闭环

2. 车牌标记准确率：YOLOv8输出车牌ROI坐标与置信度，PPOCRv3输出字符序列；ARM端将检测框与识别结果叠加显示，完成该指标且准确率高达99%以上。

注：置信度只是模型筛选参数，用来提高准确率的，并不代表准确率

3. 多角度多光照识别：已测试正常光照、强光、阴影、夜间、倾斜角度等场景，均能正常识别。

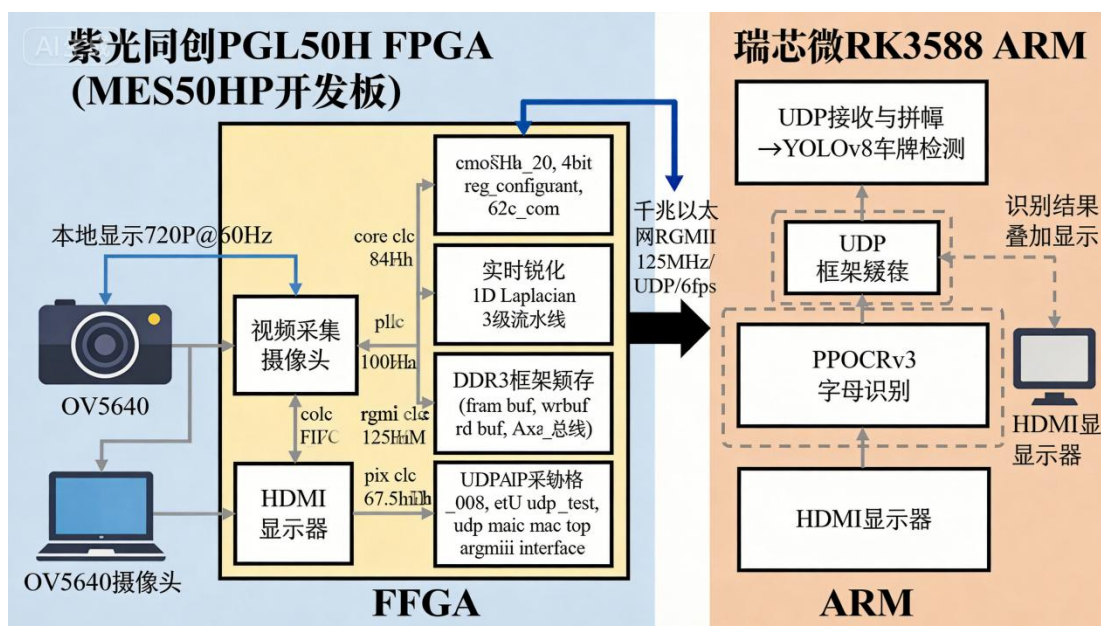
4. 识别延迟：arm端显示延迟<100ms

5. 特殊车牌识别：支持蓝牌、绿牌、双层车牌、摩托车牌、使馆领事馆、军警等多种车牌类型

2. 系统总体设计

2.1 系统硬件架构

本系统采用FPGA+ARM异构计算架构，以紫光同创PGL50H FPGA和瑞芯微RK3588 ARM处理器为核心计算单元，通过千兆以太网实现二者间的高速数据交互。系统硬件架构如图2.1所示。



FPGA端硬件平台。 FPGA选用紫光同创PGL50H-6FBG484，该芯片属于Logos系列，采用40nm工艺，具备42,800个等效LUT4、64,200个寄存器和134个DRM18K块。芯片集成4路HSST高速收发器（最高速率6.375Gbps），为光纤通信和PCIE通信提供硬件支持。配套开发板为MES50HP，采用核心板+扩展板的分层结构。

核心板主要资源配置如下：

资源类型	配置
FPGA	PGL50H-6FBG484
DDR3	2颗，32bit位宽，最高速率800Mbps×32=25,600Mbps
Flash	2颗，存储FPGA配置文件
HSST	4路，最高6.375Gbps/路

底板扩展接口配置如

下：

接口	用途	本设计使用情况
HDMI IN (MS7200)	外部视频输入	可接收，但以摄像头为主
HDMI OUT (MS7210)	本地显示输出	✅ 输出增强后的720P@60Hz视频
千兆以太网 (RTL8211E)	FPGA-ARM数据交互	✅ 发送6fps视频流至RK3588
40PIN扩展接口	OV5640摄像头连接	✅ 接入720P@30fps摄像头

ARM端硬件平台：ARM选用瑞芯微RK3588处理器，该芯片集成四核Cortex-A76与四核Cortex-A55 CPU，内置三核NPU提供6TOPS（INT8）算力，配备8GB LPDDR4内存。千兆以太网接口用于接收FPGA发送的视频流，HDMI接口用于输出叠加了识别结果的视频画面。

整体连接方式：OV5640摄像头通过40PIN排线连接FPGA开发板，FPGA开发板通过RJ45网线与RK3588直连，RK3588通过HDMI线缆连接显示器。本地监控可通过FPGA的HDMI OUT直接输出增强后的视频画面（30fps），识别结果通过RK3588的HDMI OUT输出叠加了检测框与车牌号的视频画面（6fps）。

2.2 FPGA-ARM异构分工设计

本系统核心设计思想是将任务按计算特性分配给最合适的硬件单元：FPGA擅长确定性时序控制与像素级并行处理，ARM/NPU擅长计算密集但时序灵活的AI推理。基于这一原则，系统功能分工如下。

FPGA端承担四项确定性像素级任务。

其一，图像采集。通过I2C总线配置OV5640摄像头寄存器，完成上电初始化与分辨率设定；接收8bit像素流，经cmos_8_16bit模块拼接为16bit RGB565格式。

其二，实时图像增强。通过1D锐化算法，三级流水线的架构，处理延迟固定为3个像素时钟周期（约35.7ns@84MHz）。增强后的像素同时送入DDR3帧缓存（用于本地显示）和异步FIFO（用于网络发送），如代码所示。

```
fram_buf fram_buf(
    .ddr_clk      ( core_clk      ),
    .ddr_rstn     ( ddr_init_done ),
    .vin_clk      ( pclk_in_test  ),
    .wr_fsynco    ( sharp_vs_in   ),
    .wr_en        ( sharp_de_in   ),
    .wr_data      ( sharp_rgb_in  ),

    .vout_clk     ( pix_clk       ),
    .rd_fsynco    ( vs_o          ),
    .rd_en        ( de_re         ),
    .vout_de      ( de_o          ),
    .vout_data    ( o_rgb565      ),
    .init_done    ( init_done     ),
    .axi_awaddr   ( axi_awaddr    ),
    .axi_awid     ( axi_awuser_id ),
    .axi_awlen    ( axi_awlen     ),
    .axi_awready  ( axi_awready   ),
    .axi_arvalid  ( axi_arvalid   ),
    .axi_wdata    ( axi_wdata     ),
    .axi_wstrb    ( axi_wstrb     ),
    .axi_wlast    ( axi_wuser_last),
    .axi_wready   ( axi_wready    ),
    .axi_bid      ( 4'd0          ),
    .axi_araddr   ( axi_araddr    ),
    .axi_arid     ( axi_aruser_id ),
    .axi_arlen    ( axi_arlen     ),
    .axi_arvalid  ( axi_arvalid   ),
    .axi_arready  ( axi_arready   ),
    .axi_rdata    ( axi_rdata     ),
    .axi_rvalid   ( axi_rvalid    ),
    .axi_rlast    ( axi_rlast     ),
    .axi_rid      ( axi_rid       )
);

async_fifo_l6to8 u_cam_fifo (
    .wr_clk      (pclk_in_test),
    .wr_rst      (~ddr_init_done),
    .wr_en       (sharp_de_in),
    .wr_data     (sharp_rgb_in),

    .wr_full     (),
    .wr_water_level (),
    .almost_full (fifo_row_ready),

    .rd_clk      (rgmii_clk),
    .rd_rst      (~ddr_init_done),
    .rd_en       (fifo_rd_en),
    .rd_data     (fifo_rdata),
    .rd_empty    (),
    .almost_empty ()
);
```

其三，高速数据传输。纯Verilog实现的UDP/IP/MAC三层协议栈，将图像数据以自定义封包格式封装后，通过RGMI I接口发送至RTL8211E PHY芯片。

其四，本地HDMI显示。从DDR3读取增强后的视频帧，经sync_vg模块生成1280×720@60Hz标准时序，通过MS7210编码器输出至显示器。该通路保持摄像头30fps帧率，不受网络发送帧率影响。

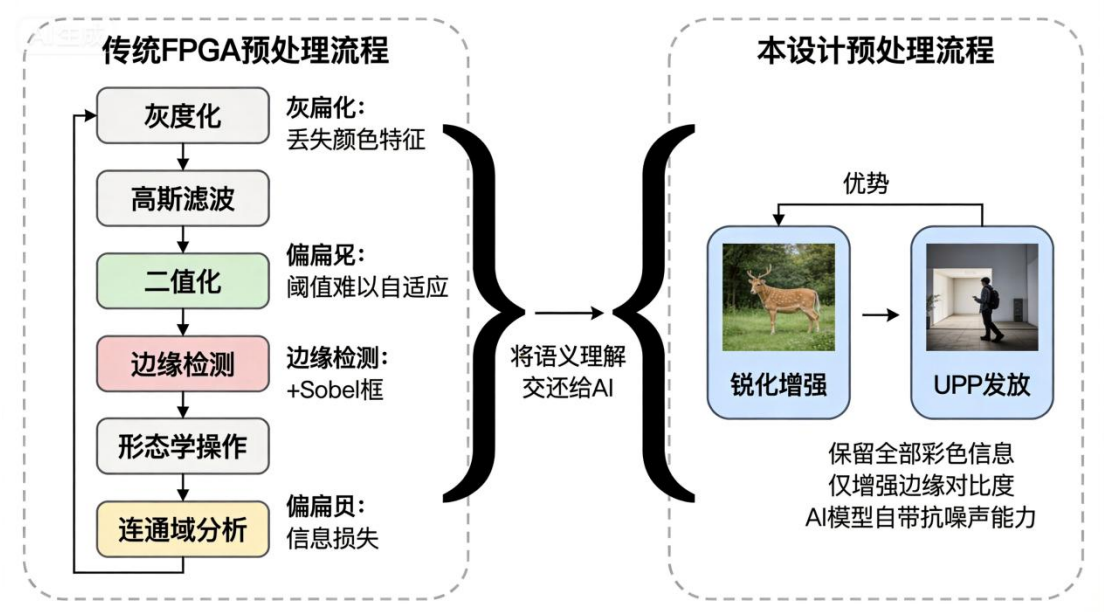
ARM端承担三项语义级AI推理任务。

其一，数据接收与帧重组。Python程序通过UDP Socket接收FPGA发送的数据包，根据帧ID与行ID字段重新拼接为完整视频帧。

其二，车牌检测。YOLOv8模型对每一帧进行推理，输出车牌区域的边界框坐标。

其三，字符识别。PP0CRv3对YOLOv8输出的车牌ROI进行字符识别，输出完整车牌号字符串，叠加显示画面。

FPGA端不做灰度化与车牌定位，赛题建议的预处理方案包括灰度化、二值化、边缘检测，但本设计有意放弃了这些步骤，如图2. X所示。



一：灰度化会丢失颜色强特征。YOLOv8在训练过程中已学习利用颜色信息区分不同类型的车牌，蓝色车牌和绿色车牌在RGB空间具有显著的色差。FPGA端灰度化将丢失颜色信息，可能降低模型准确率。

二：二值化会丢失边缘梯度信息。传统二值化依赖固定阈值，在强光、阴影、夜间等复杂光照下难以自适应。一旦阈值不当，要么丢失字符笔画，要么引入背景噪声。而YOLOv8的卷积核可学习空间梯度，原始图像中模型所需丰富的灰度过渡信息。

三：AI模型在训练阶段已使用大规模数据增强，包含高斯噪声、运动模糊、亮度/对比度扰动、透视变换等，对图像质量变化具有较强鲁棒性。额外的FPGA降噪操作反而增加硬件资源消耗。

四：在FPGA上纯硬件实现车牌定位需要组合边缘检测、形态学膨胀/腐蚀、连通域分析、颜色分割等多个算法模块，占用大量LUT和BRAM资源，多车牌场景下的区域筛选和角度校正逻辑也更为复杂。YOLOv8做定位任务既获得更高的定位精度，又能将FPGA资源集中在图像增强和数据传输等更具确定性的任务上。

2.3 通信方案选型

本设计经综合评估后选择千兆以太网UDP协议作为替代方案

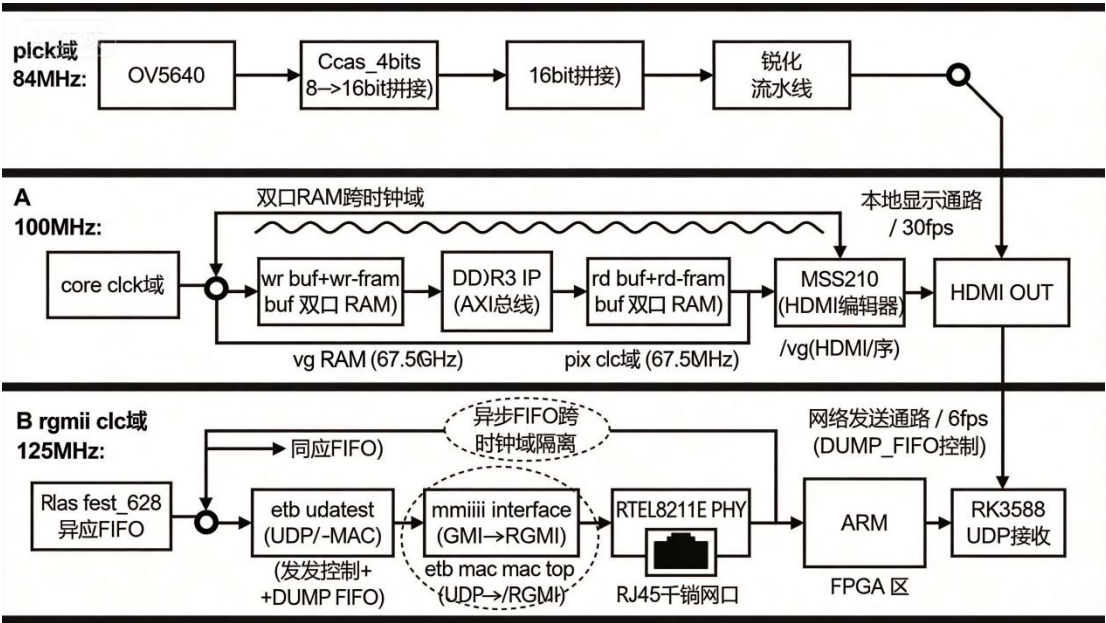
ARM端通过Python的UDP Socket接收数据。在初始联调阶段，30fps全速发送时UDP缓冲区频繁溢出，ARM端出现画面撕裂和丢包。配合帧率控制后，6fps发送使ARM端接收压力大幅降低，缓冲区不再是瓶颈。本系统中实际使用的参数为：

```
sudo sysctl -w net.core.rmem_max=1073741824
```

```
sudo sysctl -w net.core.rmem_default=524288000
```

2.4 数据流与时钟域设计

系统完整数据流与时钟域分布如图所示。



系统涉及5个独立时钟域：

时钟信号	频率	来源	用途
pclk	84MHz	OV5640输出	摄像头像素采集、锐化处理
ddrphy_clkin	100MHz	DDR3 IP核PLL	DDR3控制器与AXI总线
pix_clk	67.5MHz	FPGA内部PLL	HDMI显示时序生成
rgmii_clk	125MHz	外部PHY提供	UDP协议栈与GMII接口
sys_clk	50MHz	外部晶振	系统基础时钟、PLL输入

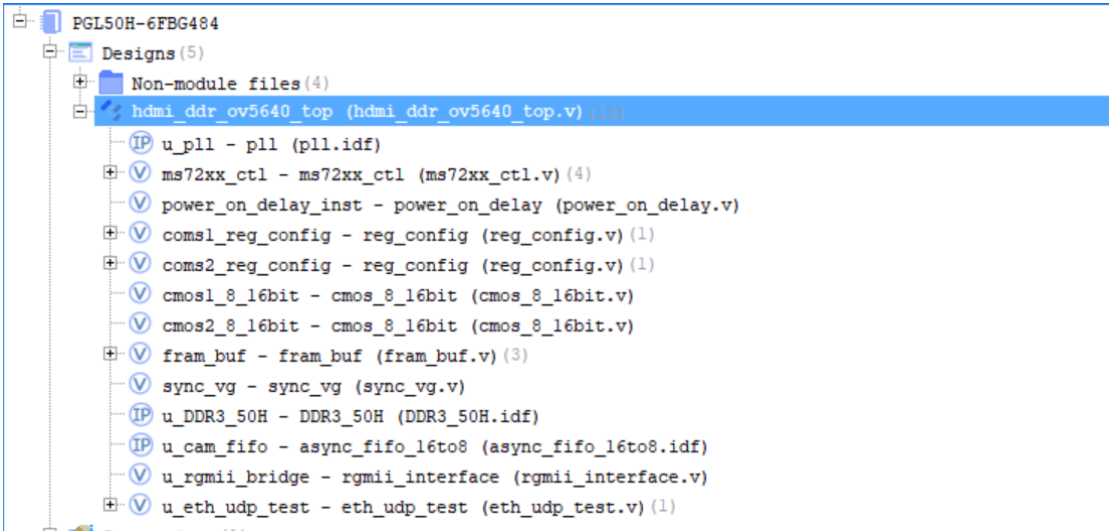
跨时钟域隔离采用两种方式：pclk与rgmii_clk之间通过async_fifo_16to8异步FIFO隔离，利用FIFO内部的双口RAM和独立的读写指针实现完全的时钟域解耦；pclk/ddrphy_clkin/pix_clk之间通过wr_buf和rd_buf内部的双口RAM（wr_fram_buf和rd_fram_buf）隔离，写端口和读端口分别工作在不同时钟域，互不干扰。

帧率分层设计。 摄像头以30fps输出视频，而RK3568端YOLOv8 NPU推理能力约为6~10fps。解决方案采用分层帧率架构：

层级	帧率	位置	实现方式
摄像头采集层	30fps	OV5640物理输出	不可变，保证硬件时序稳定
FPGA本地显示层	30fps	DDR3→HDMI输出	写读独立，不受网络通路影响
FPGA网络发送层	6fps	UDP协议栈	DUMP_FIFO每5帧取1帧发送
ARM推理层	6fps	YOLOv8+PPOCRv3	与发送层帧率匹配

2.5 模块与接口概述

FPGA端工程基于Pango Design Suite 2022.2-SP6.4开发，工程目录结构如图所示。



顶层模块：（hdmi_ddr_ov5640_top.v）实例化全部子模块，对外暴露五类物理接口：

OV5640摄像头接口

(cmosl_scl/sda/vsync/href/pclk/data/reset, 双路),

DDR3 SDRAM接口

(mem_ck/ck_n/cke/cs_n/ras_n/cas_n/we_n/odt/a/ba/dq/dqs/dqs_n/dm),

RGMII以太网接口

(rgmii_rx_ctl/rxc/rxd、rgmii_tx_ctl/txc/txd、phy_rstn),

HDMI输出接口

(pix_clk/vs_out/hs_out/de_out/r_out/g_out/b_out、iic_tx_scl/sda),

HDMI输入接口

(iic_scl/sda)。

摄像头配置模块：(power_on_delay + reg_config + i2c_com)完成OV5640三阶段上电初始化：硬件上电时序控制(PWDN低5ms→RESET低1.3ms→等待21ms)、I2C寄存器逐地址配置(20kHz I2C时钟, 357个寄存器)、配置完成标志输出。

视频采集模块：(cmos_8_16bit × 2)的核心逻辑如下图代码所示，利用帧同步信号vsync的下降沿使能像素时钟分频，将OV5640输出的8bit像素流拼接为16bitRGB565格式

```
50         always@(posedge pclk)
51         begin
52             if(!rst_n)
53                 enable <= 1'b0;
54             else if(!vs_i_reg64vs_i)
55                 enable <= 1'b1;
56             else
57                 enable <= enable;
58         end
59
60         GTP_IOCLKBUF #(
61             .GATE_EN("FALSE")//
62         ) u_GTP_IOCLKBUF (
63             .CLKOUT(pclk_IOCLKBUF), // OUTPUT
64             .CLKIN(pclk), // INPUT
65             .DI(enable) // INPUT
66         );
67
68         GTP_IOCLKDIV #(
69             .GRS_EN("TRUE"),
70             .DIV_FACTOR("2")
71         ) u_GTP_IOCLKDIV (
72             .CLKDIVOUT(pixel_clk), // OUTPUT
73             .CLKIN(pclk_IOCLKBUF), // INPUT
74             .RST_N(enable) // INPUT
75         );
76
77         always@(posedge pclk)
78         begin
79             if(!rst_n)
80                 cnt <= 2'b0;
81             else if(de_i == 1'b1 && cnt == 2'd1)
82                 cnt <= 2'b0;
83             else if(de_i == 1'b1)
84                 cnt <= cnt + 1'b1;
85         end
```

锐化增强模块：采用1D拉普拉斯算子流水线，三级流水结构：

第1级行缓冲获取左/中/右三像素，

第2级计算边缘强度与方向，

第3级加减锐化增量并钳位输出。详见3.3节。

DDR3帧缓存模块：（fram_buf + wr_buf + rd_buf + wr_rd_ctrl_top + DDR3_50H）封装DDR3读写通路，如下图所示。写通路经wr_fram_buf双口RAM跨时钟域（pclk→ddr_clk），读通路经rd_fram_buf双口RAM跨时钟域（ddr_clk→pix_clk），wr_rd_ctrl_top通过wr_cmd_trans和rd_cmd_trans模块管理AXI总线读写命令。后续会详细说明。

```
28     parameter          H_NUM          = 12'd1280,
29     parameter          V_NUM          = 12'd720,
30     parameter          PIX_WIDTH      = 16//24
31 ) (
32     input               vin_clk,
33     input               wr_fsync,
34     input               wr_en,
35     input [PIX_WIDTH- 1'b1 : 0] wr_data,
36     output reg          init_done=0,
37
38     input               ddr_clk,
39     input               ddr_rstn,
40
41     input               vout_clk,
42     input               rd_fsync,
43     input               rd_en,
44     output              vout_de,
45     output [PIX_WIDTH- 1'b1 : 0] vout_data,
46
47     output [CTRL_ADDR_WIDTH-1:0] axi_awaddr ,
48     output [3:0]             axi_awid   ,
49     output [3:0]             axi_awlen  ,
50     output [2:0]             axi_awsz   ,
51     output [1:0]             axi_awburst ,
52     input                   axi_awready ,
53     output                  axi_awvalid ,
54
55     output [MEM_DQ_WIDTH*8-1:0] axi_wdata ,
56     output [MEM_DQ_WIDTH -1 : 0] axi_wstrb ,
57     input                   axi_wlast   ,
58     output                  axi_wvalid  ,
59     input                   axi_wready  ,
60     input [3 : 0]           axi_bid    ,
61
62     output [CTRL_ADDR_WIDTH-1:0] axi_araddr ,
63     output [3:0]             axi_arid   ,
64     output [3:0]             axi_arlen  ,
65     output [2:0]             axi_arsz   ,
66     output [1:0]             axi_arburst ,
67     output                  axi_arvalid ,
68     input                   axi_arready ,
69
70     output                  axi_rready  ,
71     input [MEM_DQ_WIDTH*8-1:0] axi_rdata ,
72     input                   axi_rvalid  ,
73     input                   axi_rlast  ,
74     input [3:0]             axi_rid    ,
75 );
```

千兆以太网发送模块：由四子模块组成完整UDP发送链路，代码如下图所示。

async_fifo_16to8完成跨时钟域（pclk→rgmii_clk）与位宽转换（16bit→8bit），eth_udp_test管理发送状态机（含DUMP_FIFO帧率控制），

udp_ip_mac_top实现UDP/IP/MAC三层协议封装，rgmii_interface完成GMII到RGMII的接口转换。详见第三章。

```
async_fifo_16to8 u_cam_fifo (
    .wr_clk      (pclk_in_test),
    .wr_rst      (~ddr_init_done),
    .wr_en       (sharp_de_in),
    .wr_data      (sharp_rgb_in),

    .wr_full      (),
    .wr_water_level (),
    .almost_full   (fifo_row_ready),

    .rd_clk      (rgmii_clk),
    .rd_rst      (~ddr_init_done),
    .rd_en       (fifo_rd_en),
    .rd_data      (fifo_rdata),
    .rd_empty     (),
    .almost_empty  ()
);
wire      gmii_rx_dv;
wire [7:0] gmii_rxd;
wire      gmii_tx_en;
wire [7:0] gmii_txd;
wire      mac_rx_error;

rgmii_interface u_rgmii_bridge (
    .rst      (~ddr_init_done),
    .rgmii_clk (rgmii_clk),
    .rgmii_rxc_phase_d (rgmii_clk),
    .rgmii_clk_90p (rgmii_clk),

    .rgmii_rx_ctl (rgmii_rx_ctl),
    .rgmii_rxc    (rgmii_rxc),
    .rgmii_rxd     (rgmii_rxd),
    .rgmii_tx_ctl (rgmii_tx_ctl),
    .rgmii_txc    (rgmii_txc),
    .rgmii_txd     (rgmii_txd),

    .mac_rx_data_valid (gmii_rx_dv),
    .mac_rx_data       (gmii_rxd),
    .mac_rx_error      (mac_rx_error),
    .mac_tx_data_valid (gmii_tx_en),
    .mac_tx_data       (gmii_txd)
);

eth_udp_test u_eth_udp_test (
    .rgmii_clk (rgmii_clk),
    .rstn      (ddr_init_done),
    .gmii_tx_en (gmii_tx_en),
    .gmii_txd   (gmii_txd),
    .gmii_rx_dv (gmii_rx_dv),
    .gmii_rxd   (gmii_rxd),
    .fifo_rd_en (fifo_rd_en),
    .fifo_rdata (fifo_rdata),
    .fifo_row_ready (fifo_row_ready)
);
```

HDMI显示输出模块：（sync_vg + MS7210 I2C配置）中，sync_vg生成1280×720@60Hz的VGA标准时序（H_TOTAL=1650，H_SYNC=40，H_BP=220，H_FP=110，V_TOTAL=750，V_SYNC=5，V_BP=20，V_FP=5），同时生成de_re信号在数据有效区前2周期拉高，为DDR3读取预留响应时间。MS7210通过I2C完成HDMI编码器初始化。

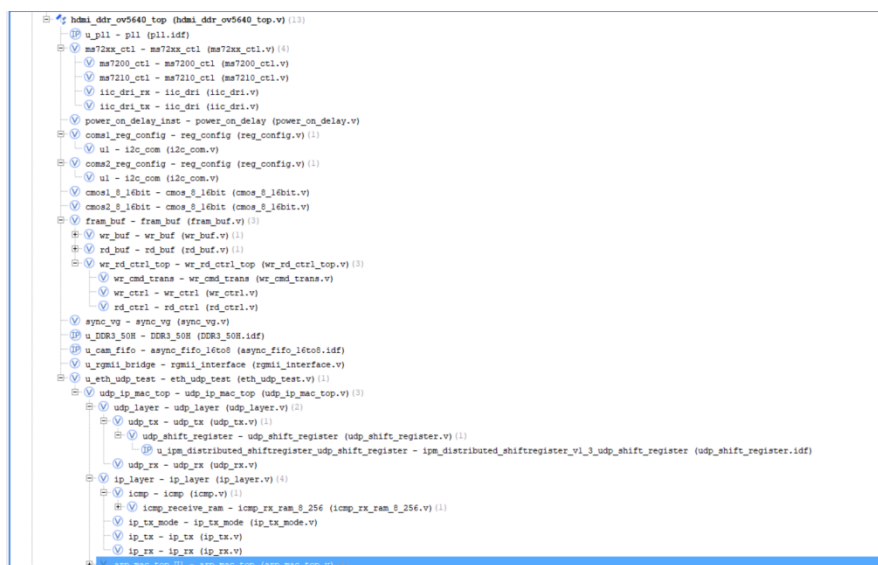
HDMI输入模块：（MS7200 + ms72xx_ctl）管理HDMI收发器的联合初始化流程，本设计中HDMI输入作为可选扩展接口保留，当前版本使用OV5640摄像头作为视频输入源。

3. FPGA端硬件设计

本系统FPGA端基于紫光同创盘古50K，Verilog开发，FPGA端承担图像采集、实时锐化增强、DDR3帧缓存、千兆以太网UDP传输、HDMI本地显示五项核心功能。各模块以流水线方式协同工作，从摄像头像素输入到网口数据包输出形成完整硬件处理链路。

3.1 FPGA工程结构与模块划分

FPGA工程采用自顶向下的层次化设计，顶层模块hdmi_ddr_ov5640_top实例化全部子模块。工程目录结构如图3.1所示。



顶层模块对外暴露五类物理接口：

接口类别	信号组	连接外部设备
摄像头接口	cmos1_scl/sda/vsync/href/pclk/data/reset（双路）	OV5640 ×2
DDR3接口	mem_ck/ck_n/cke/cs_n/ras_n/cas_n/we_n/odt/a/ba/dq/dqs/dqs_n/dm	DDR3 SDRAM
千兆以太网接口	rgmii_rx_ctl/rxc/rxd[3:0]、rgmii_tx_ctl/txc/txd[3:0]、phy_rstn	RTL8211E PHY
HDMI输出接口	pix_clk/vs_out/hs_out/de_out/r_out/g_out/b_out、iic_tx_scl/sda	MS7210编码器
HDMI输入接口	iic_scl/sda	MS7200解码器

本章按数据流顺序依次阐述各模块的硬件实现原理：视频采集→锐化增强→DDR3帧缓存→千兆以太网发送→HDMI显示输出。

3.2 多源视频采集模块

3.2.1 OV5640摄像头初始化

OV5640上电后需经过严格的时序控制和寄存器配置才能输出视频流。初始化分两个阶段完成。

硬件上电时序：OV5640的datasheet规定了明确的引脚上电顺序：先拉低PWDN保持至少5ms使传感器内部电路稳定；然后拉低RESET保持至少1ms；释放RESET后

等待至少20ms，待传感器内部PLL锁定后方可通过I2C配置寄存器。

power_on_delay模块以50MHz系统时钟为基准，通过三个计数器依次完成上述时序，代码实现如下：

```
56  always@(posedge clk_50M)begin
57      if(camera_pwnd_reg==1) begin
58          cnt2<=0;
59          camera_rstn_reg<=1'b0;
60      end
61  else if(cnt2<16'hffff) begin
62      cnt2<=cnt2+1'b1;
63      camera_rstn_reg<=1'b0;
64  end
65  else
66      camera_rstn_reg<=1'b1;
67  end
68
69  //21ms, delay from resetb pul high
70  always@(posedge clk_50M)begin
71      if(camera_rstn_reg==0) begin
72          cnt3<=0;
73          initial_en<=1'b0;
74      end
75  else if(cnt3<20'hfffff) begin
76      cnt3<=cnt3+1'b1;
77      initial_en<=1'b0;
78  end
79  else
80      initial_en<=1'b1;
81  end
```

三阶段时序满足OV5640要求：PWDN低电平持续5.24ms、RESET低电平持续1.31ms、RESET释放至I2C使能间隔20.97ms。

I2C寄存器配置：上电时序完成后，reg_config模块以生成的20kHz I2C时钟逐寄存器写入配置值。配置过程采用三段式状态机：第0拍将当前寄存器地址和数据组装为32位I2C命令字（设备地址0x78 + 寄存器地址[15:8] + 寄存器地址[7:0] + 寄存器值[7:0]）；第1拍启动I2C发送并等待tr_end传输结束标志；第2拍寄存器索引reg_index自增进入下一循环，直至357个寄存器全部配置完成。i2c_com模块负责I2C总线的具体时序——在SCL低电平时切换SDA数据，SCL高电平时SDA变化即起始/停止位，第9个SCL周期释放SDA等待从机ACK应答。

关键寄存器配置项包括：PLL倍频系数（0x3035=0x21，0x3036=0x69，设定像素时钟为84MHz）、输出开窗（0x3800-0x380F设定1280×720）、帧率控制（0x3A02-0x3A03设定曝光时间）。

3.2.2 8bit到16bit像素格式转换

OV5640输出8bit并行像素流，本系统内部统一使用16bit RGB565格式，每两个连续8bit数据构成一个完整像素。cmos_8_16bit模块完成两项工作：

像素时钟生成：模块内部实例化GTP_IOCLKBUF时钟缓冲器和GTP_IOCLKDIV二分频器，将84MHz输入像素时钟二分频为42MHz输出像素时钟。分频器以vsync帧同步信号的下降沿作为使能起点，当检测到!vs_i_reg && vs_i时拉高enble，确保每帧的像素时钟相位从同一个起点开始计数。

双像素拼接：以84MHz为工作时钟，用1bit计数器cnt区分前后半像素。cnt=0时将当前8bit数据缓存至pdata_i_reg；cnt=1时将缓存值与新输入数据拼接为16bit输出{pdata_i_reg, pdata_i}，同时产生de_o信号。

```
77     always@(posedge pclk)
78     begin
79         if(!rst_n)
80             cnt <= 2'b0;
81         else if(de_i == 1'b1 && cnt == 2'd1)
82             cnt <= 2'b0;
83         else if(de_i == 1'b1)
84             cnt <= cnt + 1'b1;
85     end
86
87
88     always@(posedge pclk)
89     begin
90         if(!rst_n)
91             pdata_i_reg <= 8'b0;
92         else if(de_i == 1'b1)
93             pdata_i_reg <= pdata_i;
94     end
95
96     always@(posedge pclk)
97     begin
98         if(!rst_n)
99             pdata_out1 <= 16'b0;
100        else if(de_i == 1'b1 && cnt == 2'd1)
101            pdata_out1 <= {pdata_i_reg,pdata_i};
102    end
103
```

拼接后的16bit数据及de_o信号经两级寄存器打拍后输出，与pixel_clk时钟域对齐。

3.2.3 视频源选择

顶层通过Verilog宏定义实现双摄像头切换，纯组合逻辑不引入额外延迟：

```
221 ④ `ifdef CMOS_1
222  assign pclk_in_test = cmos1_pclk_16bit ;
223  assign vs_in_test = cmos1_vsync_d0 ;
224  assign de_in_test = cmos1_href_16bit ;
225  assign i_rgb565 = {cmos1_d_16bit[4:0],cmos1_d_16bit[10:5],cmos1_d_16bit[15:11]};
226 ④ `elsif CMOS_2
227  assign pclk_in_test = cmos2_pclk_16bit ;
228  assign vs_in_test = cmos2_vsync_d0 ;
229  assign de_in_test = cmos2_href_16bit ;
230  assign i_rgb565 = {cmos2_d_16bit[4:0],cmos2_d_16bit[10:5],cmos2_d_16bit[15:11]};
231  `endif
```

ifdef为编译期指令不消耗额外资源。

3.3 实时锐化增强算法

3.3.1 设计动机

FPGA端的核心任务是为ARM端AI推理提供高质量图像。YOLOv8的卷积核是在学习空间梯度特征，因此FPGA预处理只需增强边缘对比度，无需灰度化，会削弱蓝绿车牌的颜色区分，或硬二值化，阈值在复杂光照下难以自适应。基于这一原则，采用1D锐化算法，仅增强图像边缘，完整保留彩色信息。

3.3.2 算法原理

算法基于1D拉普拉斯算子的变体。对于水平方向三个相邻像素——左像素p_l、中心像素p_c、右像素p_r——定义：

边缘强度 $edge = p_c \times 2 - (p_l + p_r)$

当 $edge > 0$ 时，中心比左右更亮（亮边缘），需进一步提亮中心像素；

当 $edge < 0$ 时，中心比左右更暗（暗边缘），需进一步压暗。锐化输出为：

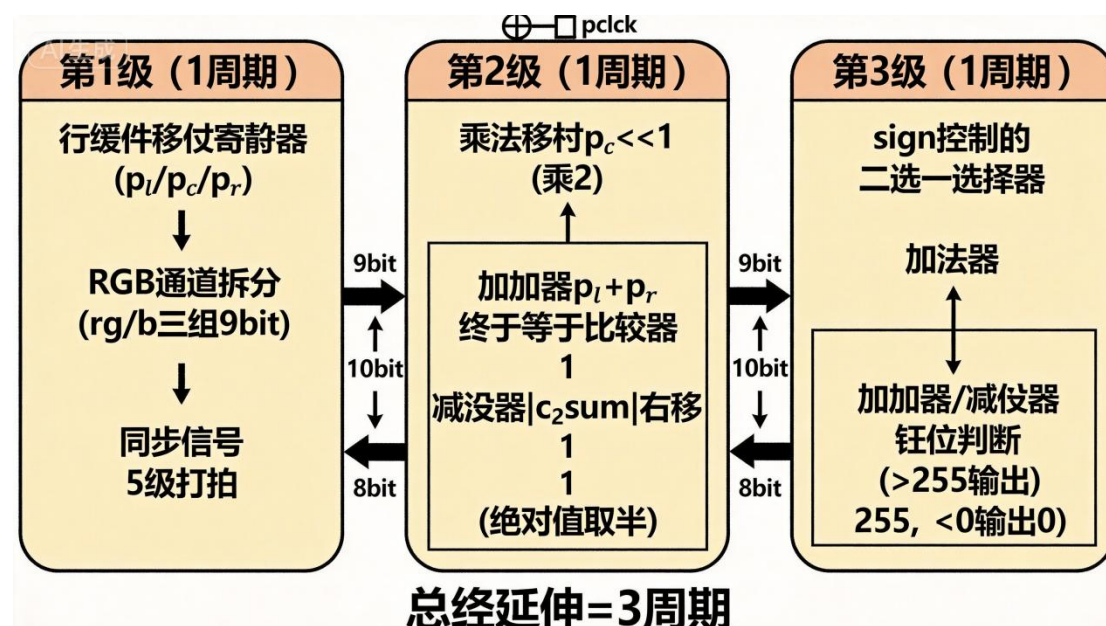
$p_{out} = clamp(p_c + edge/2)$ （亮边缘，提亮）

$p_{out} = clamp(p_c - |edge|/2)$ （暗边缘，压暗）

其中除以2通过右移1位实现，clamp将结果钳位在0~255范围。50%的锐化力度（÷2）在增强边缘的同时保留图像自然感，避免过度锐化引入振铃效应。

3.3.3 三级流水线硬件实现

算法以三级流水线结构实现，每级严格占用1个时钟周期，从像素输入到锐化输出固定3周期延迟。流水线结构如图所示。



第1级：行缓冲与像素提取。

用两级移位寄存器构建行缓冲器，同时获取左 (p_l)、中 (p_c)、右 (p_r) 三个像素。同步信号 (de_in , vs_in) 通过5级移位寄存器打拍对齐，确保第5拍输出时像素数据和同步信号严格对应。输入16bit RGB565按通道拆分为3组9bit 无符号数——高位补1bit 0，防止后续乘2溢出。

```

241 always @(posedge pclk_in_test) begin
242     if (!ddr_init_done) begin
243         p_l <= 0; p_c <= 0; p_r <= 0;
244         de_pipe <= 0; vs_pipe <= 0;
245     end else begin
246         if (de_in_test) begin
247             p_l <= p_c; p_c <= p_r; p_r <= i_rgb565;
248         end else begin
249             p_l <= 0; p_c <= 0; p_r <= 0;
250         end
251         de_pipe <= {de_pipe[3:0], de_in_test};
252         vs_pipe <= {vs_pipe[3:0], vs_in_test};
253     end
254 end
255
256 // 扩展到 9 位无符号数 (最大值255, 9位绝对不溢出)
257 wire [8:0] r_l = {1'b0, p_l[15:11], 3'b0}; wire [8:0] g_l = {1'b0, p_l[10:5], 2'b0}; wire [8:0] b_l = {1'b0, p_l[4:0], 3'b0};
258 wire [8:0] r_c = {1'b0, p_c[15:11], 3'b0}; wire [8:0] g_c = {1'b0, p_c[10:5], 2'b0}; wire [8:0] b_c = {1'b0, p_c[4:0], 3'b0};
259 wire [8:0] r_r = {1'b0, p_r[15:11], 3'b0}; wire [8:0] g_r = {1'b0, p_r[10:5], 2'b0}; wire [8:0] b_r = {1'b0, p_r[4:0], 3'b0};

```

第2级：边缘强度计算。

对三个通道分别计算：中心 $\times 2$ ($p_c \ll 1$)、左右和 ($p_l + p_r$)。比较中心 $\times 2$ 与左右和的大小决定边缘方向 ($sign$)，计算两者绝对差值后右移1位 ($\div 2$) 作为锐化增量。所有中间量扩展为10bit，防止求和及乘2溢出。

```

266 always @(posedge pclk_in_test) begin
267     sum_r <= r_l + r_r; c2_r <= r_c << 1; cdl_r <= r_c;
268     sum_g <= g_l + g_r; c2_g <= g_c << 1; cdl_g <= g_c;
269     sum_b <= b_l + b_r; c2_b <= b_c << 1; cdl_b <= b_c;
270 end
271
272 // 3. 第二级流水：比较大小，计算绝对差值的一半 (>>1 即50%力度)
273 reg sign_r, sign_g, sign_b;
274 reg [8:0] half_det_r, half_det_g, half_det_b;
275 reg [8:0] cd2_r, cd2_g, cd2_b;
276 always @(posedge pclk_in_test) begin
277     // ★ sign=0 表示中心离边缘 (需提高), sign=1 表示暗边缘 (需压暗)
278     // 绝对不会产生任何负数下溢出!
279     sign_r <= (c2_r >= sum_r) ? 1'b0 : 1'b1;
280     half_det_r <= ((c2_r >= sum_r) ? (c2_r - sum_r) : (sum_r - c2_r)) >> 1;
281     cd2_r <= cdl_r;
282
283     sign_g <= (c2_g >= sum_g) ? 1'b0 : 1'b1;
284     half_det_g <= ((c2_g >= sum_g) ? (c2_g - sum_g) : (sum_g - c2_g)) >> 1;
285     cd2_g <= cdl_g;
286
287     sign_b <= (c2_b >= sum_b) ? 1'b0 : 1'b1;
288     half_det_b <= ((c2_b >= sum_b) ? (c2_b - sum_b) : (sum_b - c2_b)) >> 1;
289     cd2_b <= cdl_b;
290 end

```

第3级：加减锐化与钳位。

根据sign决定加或减锐化增量。加法时检查`res > 255`则取255，减法时检查`cd2 < half_det`则取0（防止负数下溢出），其余取`res[7:0]`。钳位完全采用if判断，不依赖任何补码截位操作。三通道重新组合为16bit RGB565输出。

```

293 reg [7:0] out_r, out_g, out_b;
294 wire [9:0] res_r_add = cd2_r + half_det_r;
295 wire [9:0] res_r_sub = cd2_r - half_det_r;
296 wire [9:0] res_g_add = cd2_g + half_det_g;
297 wire [9:0] res_g_sub = cd2_g - half_det_g;
298 wire [9:0] res_b_add = cd2_b + half_det_b;
299 wire [9:0] res_b_sub = cd2_b - half_det_b;
300
301 always @(posedge pclk_in_test) begin
302     if (!ddr_init_done) begin
303         out_r <= 0; out_g <= 0; out_b <= 0;
304
305         out_r <= (sign_r == 1'b0) ? ((res_r_add > 255) ? 8'd255 : res_r_add[7:0]) : ((cd2_r < half_det_r) ? 8'd0 : res_r_sub[7:0]);
306         out_g <= (sign_g == 1'b0) ? ((res_g_add > 255) ? 8'd255 : res_g_add[7:0]) : ((cd2_g < half_det_g) ? 8'd0 : res_g_sub[7:0]);
307         out_b <= (sign_b == 1'b0) ? ((res_b_add > 255) ? 8'd255 : res_b_add[7:0]) : ((cd2_b < half_det_b) ? 8'd0 : res_b_sub[7:0]);
308     end
309 end

```

3.3.4 无符号数设计的鲁棒性

Verilog中signed数据类型在不同综合器间的行为差异是数字设计的常见陷阱。`$signed()`声明的信号在移位、截位和比较操作中可能因综合器对补码扩展的实现不同而产生非预期结果。本设计从架构层面规避了：全部运算采用无符号数声明；正负号判断用显式的物理大小比较（`if(c2 >= sum)`）替代符号位判断；所有加减法操作数至少保留1bit高位余量防止溢出；钳位保护通过纯if判断实现，不涉及任何补码截位。确保了跨综合器的完全兼容性。

3.3.5 资源开销

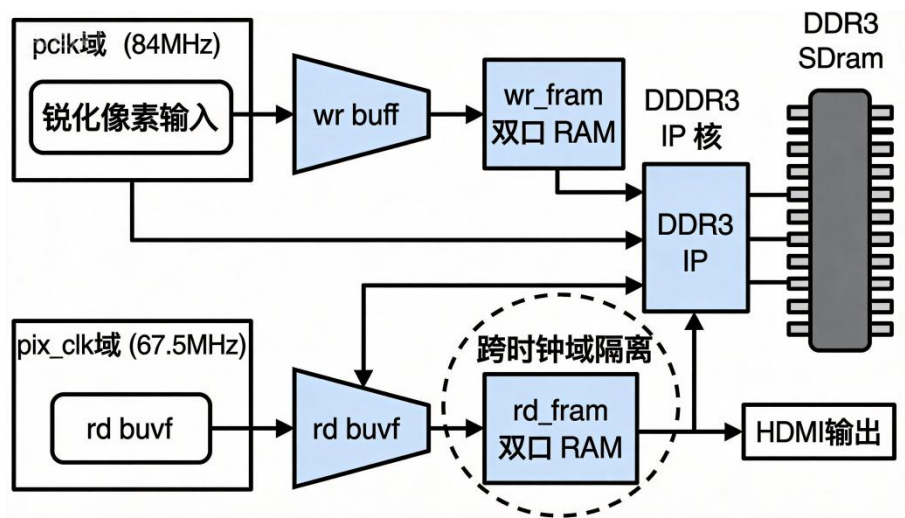
锐化模块硬件消耗低：所有乘法用移位实现，使DSP块零占用，行缓冲用分布式寄存器实现使BRAM块零占用。在FPGA整体LUT占用率17.39%的背景下，锐化模块占比不到3%，低硬件成本换取了图像边缘质量的提升。

3.4 DDR3帧缓存与读写控制

3.4.1 整体架构

DDR3控制器采用紫光同创DDR3_50H IP核，通过AXI总线接口与用户逻辑交互。IP核配置：ROW_ADDR_WIDTH=15，COL_ADDR_WIDTH=10，BANK_ADDR_WIDTH=3，DQ_WIDTH=32bit。IP核内部自动完成DDR3物理时序管理（刷新、激活、预充电、读写操作），对外屏蔽复杂的SDRAM时序细节。

fram_buf模块封装完整读写通路，对外提供两套独立时钟域接口，如图所示。



```

28     parameter          H_NUM          = 12'd1280,
29     parameter          V_NUM          = 12'd720,
30     parameter          PIX_WIDTH      = 16//24
31 ) (
32     input               vin_clk,
33     input               wr_fsync,
34     input               wr_en,
35     input [PIX_WIDTH- 1'b1 : 0] wr_data,
36     output reg          init_done=0,
37
38     input               ddr_clk,
39     input               ddr_rstn,
40
41     input               vout_clk,
42     input               rd_fsync,
43     input               rd_en,
44     output              vout_de,
45     output [PIX_WIDTH- 1'b1 : 0] vout_data,
46
47     output [CTRL_ADDR_WIDTH-1:0] axi_awaddr ,
48     output [3:0]             axi_awid   ,
49     output [3:0]             axi_awlen  ,
50     output [2:0]             axi_awsiz  ,
51     output [1:0]             axi_awburst,
52     input                   axi_awready ,
53     output                   axi_awvalid ,
54
55     output [MEM_DQ_WIDTH*8-1:0] axi_wdata ,
56     output [MEM_DQ_WIDTH -1 :0] axi_wstrb ,
57     input                   axi_wlast  ,
58     output                   axi_wvalid ,
59     input                   axi_wready ,
60     input [3 : 0]           axi_bid    ,
61
62     output [CTRL_ADDR_WIDTH-1:0] axi_araddr ,
63     output [3:0]             axi_arid   ,
64     output [3:0]             axi_arlen  ,
65     output [2:0]             axi_arsiz  ,
66     output [1:0]             axi_arburst,
67     output                   axi_arvalid ,
68     input                   axi_arready ,
69
70     output                   axi_rready ,
71     input [MEM_DQ_WIDTH*8-1:0] axi_rdata ,
72     input                   axi_rvalid  ,
73     input                   axi_rlast  ,
74     input [3:0]             axi_rid    ,
75 );

```

3.4.2 写通路设计

写通路数据流：锐化像素 → wr_buf内部行缓存（wr_fram_buf双口RAM，位宽32bit，深度512）→ wr_cmd_trans命令转换 → wr_ctrl AXI写通道控制 → DDR3 IP核。

wr_buf模块处理跨时钟域和位宽匹配。写入侧在pc1k域完成行缓存，以wr_fsync帧同步信号触发新一帧的写入起始地址计算；读出侧在core_clk域通过rd_bac（back-pressure）信号控制读取速率，防止AXI总线拥塞。数据以整行为单位写入DDR3，每行一次AXI Burst传输。

wr_ctrl实现标准AXI写通道握手协议：当wr_en有效时，依次发起写地址命令（awvalid与awready握手）和写数据命令（wvalid与wready握手）；等待写响应（bvalid与bready握手）后判定一次Burst传输完成。

3.4.3 读通路设计

读通路数据流：rd_buf发起读命令 → rd_cmd_trans命令转换 → rd_ctrl AXI读通道控制 → DDR3 IP核返回读数据 → rd_fram_buf双口RAM缓存 → vout_clk域读取。

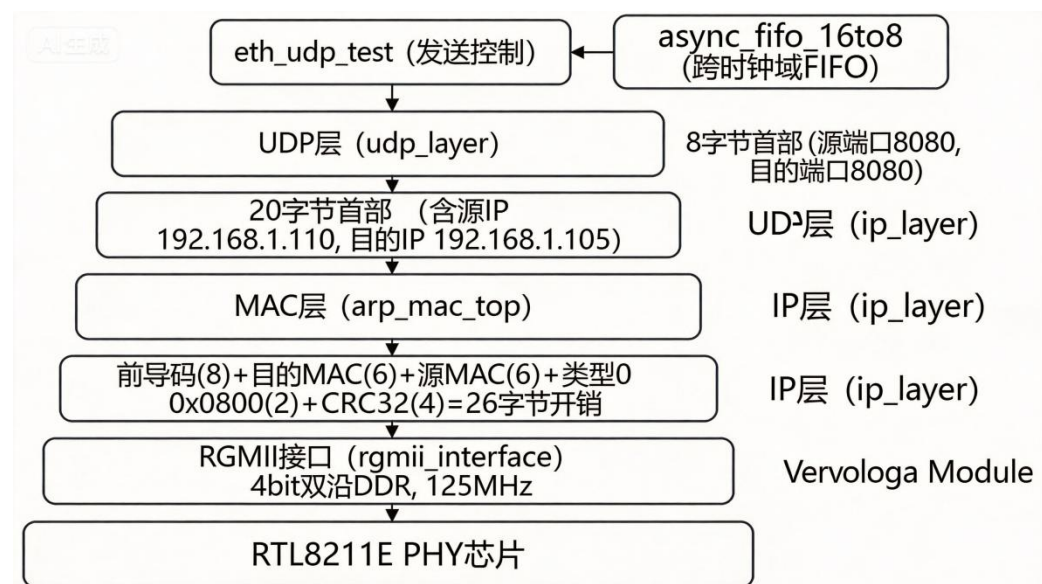
`rd_ctrl`实现标准AXI读通道握手：`read_en`有效时发起读地址命令（`arvalid`与`arready`握手）；等待读数据返回（`rvalid`与`rready`握手），数据逐拍存入`rd_fram_buf`。读数据以整行为单位从DDR3读取。

3.4.4 跨时钟域处理

写通路和读通路各使用一个双口RAM作为跨时钟域隔离。双口RAM的写端口和读端口完全独立，各自工作在所属时钟域，二者之间仅共享RAM存储阵列。写端口和读端口无需任何握手信号，完全解耦。双口RAM的建立/保持时间由FPGA片内BRAM硬核保证，时序收敛可靠，不消耗额外的跨时钟域同步寄存器。

3.5 千兆以太网传输模块

千兆以太网传输模块是FPGA与ARM之间的数据桥梁。本设计以纯Verilog实现完整UDP/IP/MAC三层硬件协议栈。数据封装过程自上而下为：应用数据（8字节自定义头+1280字节像素）→ 添加UDP首部（8字节）→ 添加IP首部（20字节）→ 添加MAC首部（14字节）+ CRC32（4字节）→ 通过RGMII接口发出。协议层次结构如图所示。



各协议层功能与对应Verilog模块：

协议层	模块名	功能
应用层（发送控制）	eth_udp_test	状态机管理，封包组织，帧率控制
传输层（UDP）	udp_layer	8字节UDP首部，校验和
网络层（IP）	ip_layer	20字节IP首部，首部校验和
数据链路层（MAC+ARP）	arp_mac_top	14字节MAC首部+4字节CRC32
物理接口层	rgmii_interface	GMII $\rightleftharpoons$ RGMII转换

3.5.1 GMII接口与RGMII桥接

千兆以太网MAC层与PHY芯片的标准接口为GMII（8bit数据总线、125MHz时钟、单沿采样）。但多数PHY芯片（包括本设计使用的RTL8211E）物理引脚为RGMII（4bit数据总线、125MHz时钟、双沿DDR采样），以一半的信号线数量实现相同带宽。

rgmii_interface模块完成GMII到RGMII的接口转换。发送方向：在rgmii_txc上升沿输出gmii_txd[3:0]，下降沿输出gmii_txd[7:4]；接收方向：在rgmii_rxc双沿采样rgmii_rxd[3:0]，拼接为8bit数据输出。

RTL8211E PHY芯片的RGMII信号与FPGA IO引脚的连接关系：

FPGA端信号	方向	PHY端引脚	功能
rgmii_txc	output	TXC	125MHz发送时钟
rgmii_tx_ctl	output	TX_CTL	发送使能
rgmii_txd[3:0]	output	TXD[3:0]	发送数据（双沿DDR）
rgmii_rxc	input	RXC	125MHz接收时钟（PHY提供）
rgmii_rx_ctl	input	RX_CTL	接收有效
rgmii_rxd[3:0]	input	RXD[3:0]	接收数据（双沿DDR）

MES50HP开发板板载RTL8211E PHY芯片，上述RGMII信号在PCB上已直连至PGL50H FPGA对应IO引脚。RJ45网口通过标准CAT5e/CAT6网线连接至RK3588开发板的千兆网口，FPGA（192.168.1.110）与RK3588（192.168.1.105）配置为同一子网，无需路由器，在数据链路层直接通信。

3.5.2 异步FIFO跨时钟域设计

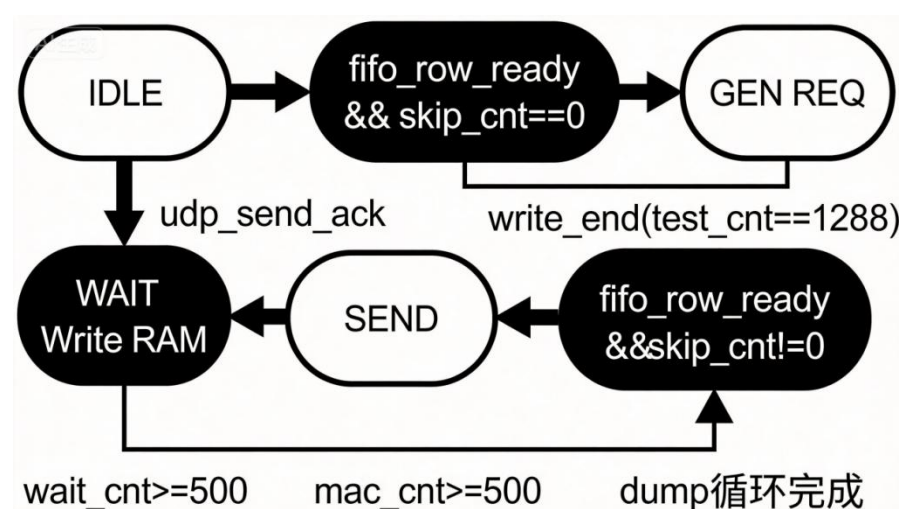
`async_fifo_16to8`模块连接摄像头像素时钟域（`pclk`，84MHz）和RGMII时钟域（`rgmii_clk`，125MHz），同时完成位宽转换（16bit→8bit）。写端口接收锐化后的RGB565像素，读端口输出字节流供UDP封包使用。

该模块的`almost_full`水线信号是整个UDP发送链路的触发起点。当FIFO内数据量达到一行像素（1280像素 = 2560字节）时，`almost_full`拉高，连接至`eth_udp_test`的`fifo_row_ready`输入，触发一次发送流程。该机制保证每次发送的恰好是一行完整像素数据，避免半行发送导致的拼帧错位。

读端时钟（125MHz）快于写端时钟（42MHz，16bit折算后等效84MHz），FIFO不存在溢出风险。

3.5.3 发送控制状态机

`eth_udp_test`模块负责管理整个UDP发送流程，采用6状态有限状态机，状态转移如图所示。



各状态功能：

状态	功能	触发与退出条件
IDLE	空闲，等待行就绪	fifo_row_ready=1且skip_cnt=0→GEN_REQ; skip_cnt≠0→DUMP_FIFO
GEN_REQ	向协议栈发起UDP发送请求	udp_send_ack=1→WRITE_RAM
WRITE_RAM	逐字节写入1288字节数据	test_cnt==1288→SEND
SEND	等待协议栈完成物理发送	mac_send_end=1→WAIT
WAIT	包间间隔500周期（约4 μs）	wait_cnt>=500→IDLE
DUMP_FIFO	读空FIFO后丢弃，不发送	1280字节读完后→IDLE

WRITE_RAM状态内的数据组织。 该状态下test_cnt从0计数到1287，每个周期写入1字节。前8字节（test_cnt=0~7）为自定义图像元数据包头，后1280字节（test_cnt=8~1287）为从FIFO读出的像素数据，如代码所示：

```

192
193
194         if (test_cnt == 16'd0) ram_wr_data <= frame_id[15:8];
195         else if (test_cnt == 16'd1) ram_wr_data <= frame_id[7:0];
196         else if (test_cnt == 16'd2) ram_wr_data <= row_id[15:8];
197         else if (test_cnt == 16'd3) ram_wr_data <= row_id[7:0];
198         else if (test_cnt == 16'd4) ram_wr_data <= {7'b0, split_id};
199         else if (test_cnt == 16'd5) ram_wr_data <= (row_id == 0 && split_id == 0) ? 8'h01 : 8'h00;
200         else if (test_cnt == 16'd6) ram_wr_data <= (row_id == 719 && split_id == 1) ? 8'h01 : 8'h00;
201         else if (test_cnt == 16'd7) ram_wr_data <= 8'h00;
202         else begin
                ram_wr_data <= fifo_rdata;
        end

```

WAIT状态的帧间间隔。 以太网标准要求相邻帧间至少96bit时间（96ns@1Gbps）的帧间间隙。本设计等待500个125MHz周期（约4 μs），远超标准最小要求，为PHY芯片和交换机提供充裕的处理时间，避免缓冲区溢出。

3.5.4 自定义图像封包格式

每个UDP数据包携带1288字节载荷——8字节自定义图像元数据+1280字节像素数据。720P一行有1280个像素，每个16bit RGB565像素展开为2字节，一行共

2560字节。2560字节无法装入单个UDP包（受限于以太网MTU 1500字节），因此将一行拆分为两个UDP包发送：

分段	split_id	携带像素范围	字节数	row_start	row_end
前半行	0	像素0~639	1280字节像素	1	0
后半行	1	像素640~1279	1280字节像素	0	1

完整封包格式：

字节偏移	字段	位宽	说明
0-1	frame_id	16bit	帧序号，ARM端据此判断帧切换
2-3	row_id	16bit	行号，0~719
4	split_id	1bit	分段标志（0=前半行，1=后半行）
5	row_start	1bit	行首标志（仅前半行拉高）
6	row_end	1bit	行尾标志（仅后半行拉高）
7	reserved	8bit	保留字段
8-1287	pixel_data	1280字节	640像素RGB565原始数据

一帧720行共计1440个UDP数据包。ARM端根据frame_id和row_id完成图像拼帧，split_id区分前后半行，row_start和row_end标记各行边界。

3.5.5 UDP/IP/MAC三层协议封装

协议栈按照TCP/IP标准在用户数据外层逐层添加协议首部。

UDP层（udp_layer）：添加8字节UDP首部：源端口(8080)、目的端口(8080)、UDP总长度(1296字节)、校验和（覆盖UDP首部+数据+IP伪首部）。模块在收到app_data_request后拉高udp_send_ack握手，逐字节接收1288字节应用数据，缓存后添加UDP首部交给IP层。

IP层（ip_layer）：添加20字节IP首部，关键字段：版本号=4、首部长度=5（20字节）、总长度=UDP数据报长度+20、标识符每包递增、不分片（DF=1）、TTL=64、协议=17（UDP）、首部校验和（16位反码求和，仅校验IP首部）、源IP=192.168.1.110、目的IP=192.168.1.105。

MAC层 (arp_mac_top)： 内部集成MAC发送 (mac_tx) 和接收 (mac_rx) 逻辑。MAC发送状态机在收到IP层数据后依次发送：8字节前导码 (7×0x55+0xD5，接收端时钟同步用)、6字节目的MAC地址、6字节源MAC地址、2字节帧类型 (0x0800表示IPv4)、IP数据报 (46~1500字节)、4字节CRC32校验 (对从目的MAC到数据尾的全部字段计算)。

ARP功能说明： FPGA与RK3588为网线直连，双方IP和MAC地址已知且固定，因此本设计关闭ARP动态地址解析 (arp_request_req=1'b0)，简化协议栈状态机。若需接入局域网，仅需拉高该信号即可启用标准ARP协议。

3.5.6 帧率控制：DUMP_FIFO机制

问题背景： 摄像头以30fps固定帧率输出，而RK3588端YOLOv8 NPU推理能力约6~10fps。若FPGA全速30fps发送，ARM端UDP缓冲区迅速打满，导致画面延迟、丢包和撕裂。

方案迭代： 第一次尝试：在WRITE_RAM状态内判断当前帧需丢弃时拉低ram_wr_en，不向网卡写入数据。但网卡IP核底层状态机已被前一包唤醒，正等待接收固定1288字节数据——突然停止写入导致状态机死锁，PC端完全收不到包。第二次尝试：在顶层hdmi_ddr_ov5640_top.v中于FIFO写入端增加net_fifo_wr_en控制，在需丢弃的4帧期间将锐化数据阻挡在FIFO外。但UDP模块完全依赖FIFO的almost_full水线信号触发发送——源头阻断导致FIFO内数据量永远达不到水线阈值，UDP状态机永远停在IDLE，剩余的已缓存数据死死卡在FIFO里出不来，帧同步和行号就此错乱。

最终方案：

核心思想： 让摄像头全速写FIFO，让FIFO永远满血运转，只在FIFO出口做丢弃。具体实现：维护skip_cnt计数器 (0→1→2→3→4→0循环，以fifo_row_ready为触发递增)。skip_cnt=0时走正常发送路径 (IDLE→GEN_REQ→WRITE_RAM→SEND→WAIT)，数据写入协议栈发送至网口。skip_cnt=1-4时进入DUMP_FIFO状态——仅从FIFO读出1280字节数据后直接丢弃，不唤醒UDP协议栈，网卡不发生任何发送行为。

DUMP_FIFO状态下的FIFO读出操作与正常发送完全一致，都从FIFO取走1280字节数据，区别仅在于数据去向：正常发送时写入ram_wr_data送入协议栈，DUMP_FIFO时不写入任何模块直接被丢弃。这保证了每行数据FIFO都会被清空一次，流水线机制持续正常工作；网卡IP核仅在skip_cnt=0时被唤醒，其余4帧完全不被影响，握手时序安全。

DUMP_FIFO机制带来的直接效果：网络带宽从全速30fps时的约400Mbps断崖式降至6fps时的约80Mbps；ARM端收到的每一帧都是完整、无撕裂的图像；本地DDR3→HDMI显示通路保持30fps不变，本地监控不受影响。

3.6 HDMI显示输出模块

3.6.1 VGA/HDMI时序生成

sync_vg模块生成1280×720@60Hz标准VESA时序，按如下参数配置：

参数	水平方向（像素）	垂直方向（行）
有效区域（Active）	1280	720
同步脉冲（Sync）	40	5
后廊（Back Porch）	220	20
前廊（Front Porch）	110	5
总周期（Total）	1650	750

模块内部维护两个计数器：h_count在像素时钟（67.5MHz）驱动下从0到1649循环计数；v_count在h_count==1649时递增。hs_out在h_count<40时拉高，vs_out在v_count<5时拉高，de_out在两计数器均处于有效区域时拉高。

一个关键的时序设计是`de_re`信号——在`de_out`有效区域前2个像素周期提前拉高。`de_re`连接至`fram_buf`的读端口作为DDR3读取触发信号，提前2周期为AXI总线读写命令和DDR3物理读取预留响应时间，确保HDMI显示数据流不中断。

3.6.2 HDMI编码器配置与像素格式转换

MS7210 HDMI编码器通过I2C总线进行初始化配置。`ms72xx_ctl`模块统一管理MS7200（HDMI输入解码器）和MS7210（HDMI输出编码器）的联合初始化——先配置MS7200使其进入稳定状态，再配置MS7210完成编码器寄存器设定。

`iic_dri`模块复用于MS7210的I2C通信，与摄像头共用同一套I2C驱动逻辑，仅设备地址和寄存器列表不同。

从DDR3读出的16bit RGB565像素在输出前转换为24bit RGB888

转换后的RGB888数据与`sync_vg`生成的`vs_out`、`hs_out`、`de_out`同步输出至MS7210编码器，最终通过HDMI接口输出1280×720@60Hz标准视频信号。

4. ARM端软件设计

本系统ARM端基于瑞芯微RK3588处理器，利用其内置NPU完成AI推理任务。ARM端承担三项核心功能：通过千兆以太网UDP协议接收FPGA发送的RGB565图像数据并完成帧重组；部署YOLOv8模型进行车牌区域检测；部署PP0CRv4模型进行车牌字符识别。识别结果通过HDMI接口输出至显示器，叠加显示检测框与车牌号字符串。

系统主程序`plate_ok.py`完整实现了上述功能流程。

4.1 系统总体流程

ARM端主程序按以下顺序完成初始化与持续运行：

模型加载阶段：程序依次加载YOLOv8车牌检测模型（`best.rknn`）、PP0CRv4文字检测模型（`ppocrv4_det.rknn`）和PP0CRv4文字识别模型

(ppocrv4_rec.rknn)。三个模型均通过RKNNLite API加载至NPU运行时，推理引擎自动将计算任务调度到三核NPU上并行执行。

UDP初始化阶段：创建UDP Socket绑定至指定端口（默认32896），设置8MB接收缓冲区。程序以0.1秒超时轮询方式接收数据包，兼顾实时性与CPU占用。

帧重组与识别阶段：每从FPGA收到一个UDP包（1288字节），解析包头中的frame_id、row_id和split_id字段，将640像素RGB565数据转换为BGR格式填入帧缓冲区的对应行和列位置。代码省略了row_start和row_end的显式校验，直接通过row_id和split_id组合判断一行是否完整——row_id=719且split_id=1表示一帧的最后一包，触发该帧的车牌识别处理。程序采取每5帧识别一次的策略以降低计算压力，跳帧期间复用上一次的标注结果用于显示。

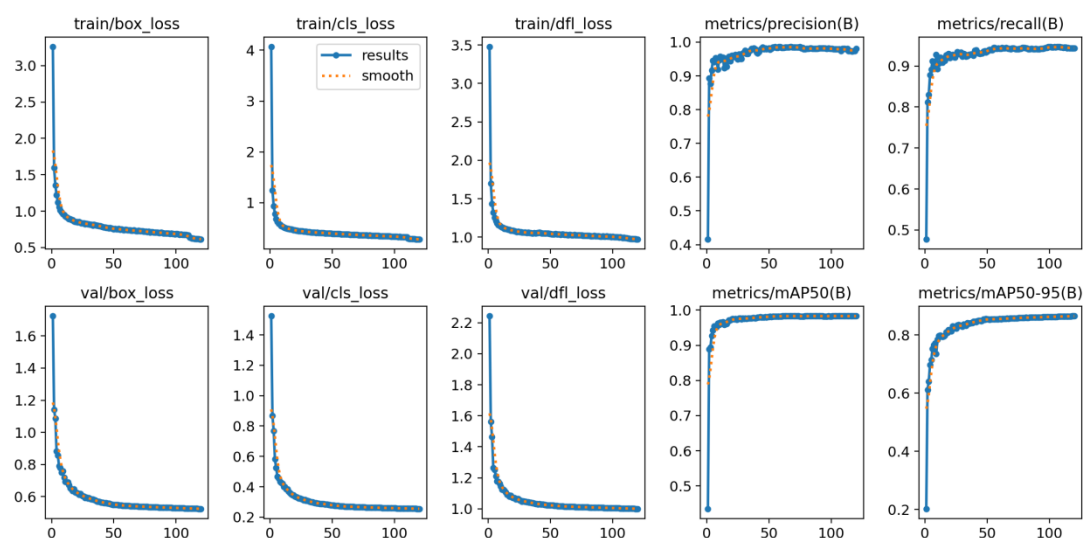
```
if row_id == 719 and split_id == 1:
    frame_count += 1
    if frame_count % DETECT_EVERY_N_FRAMES == 1 or frame_count == 1:
        annotated, results = process_frame_for_plates(
            frame_buf, rknn_yolo, ppocr_rec)
```

4.2 YOLOv8车牌检测

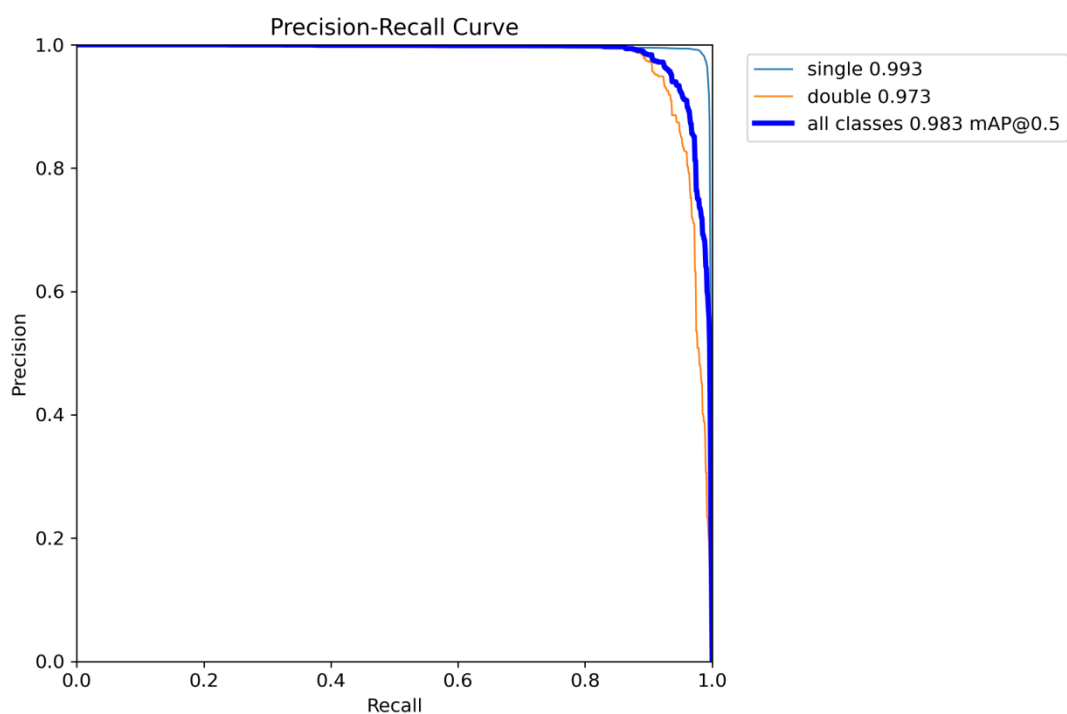
模型推理流程：接收到的完整帧图像（720×1280）首先经Letterbox处理缩放至640×640并填充灰边，再送入NPU进行YOLOv8推理。YOLOv8输出三个尺度的特征图，后处理函数yolo_post_process完成以下步骤：对三个分支分别进行DFL（Distribution Focal Loss）解码和边界框坐标还原；将所有检测框展平合并；根据置信度阈值（OBJ_THRESH=0.25）和类别得分筛选候选框；对每类候选框执行NMS（NMS_THRESH=0.45）去重。

训练效果：YOLOv8模型以单行车牌（single）和双行车牌（double）两类为目标进行训练，训练曲线如图4.1所示。从图中可以看出：前100个epoch内各项损

失指标快速收敛；mAP50在整个训练过程中稳步提升，最终达到约0.98的水平。



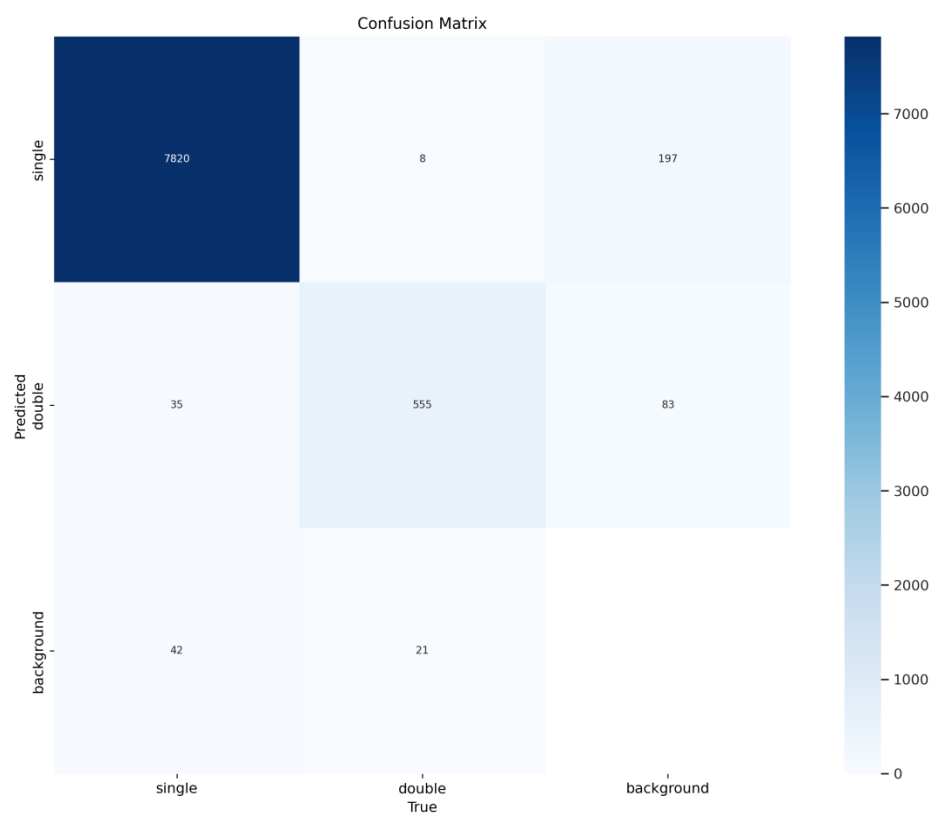
图中包含训练损失曲线（box_loss/cls_loss/df1_loss，呈稳定下降趋势）、验证指标曲线（metrics/mAP50(B)/metrics/mAP50-95(B)，呈稳定上升趋势）和验证损失曲线（val/box_loss/val/cls_loss/val/df1_loss，与训练损失同步下降）。纵轴为损失值或mAP值，横轴为epoch数（0~100）。PR曲线（Precision-Recall Curve）如图4.2所示。单行车牌（single）的AP值达到0.993，双行车牌（double）的AP为0.973，整体mAP@0.5为0.983。高mAP值表明模型在精确率和召回率之间取得了良好平衡，两种车牌类型均具有较高的检测可靠性。



图中横轴为召回率（Recall），纵轴为精确率（Precision），两条曲线分别对应single（标注AP=0.993）和double（标注AP=0.973），粗线为all classes（标注mAP@0.5=0.983）。

混淆矩阵如图，矩阵显示：单行车牌（single）被正确识别的样本数为7820个，误识别为双行车牌（double）197个，误识别为背景42个；双行车牌（double）被正确识别的样本数为555个，误识别为单行车牌35个，误识别为背景21个。双行车牌存在一定比例的漏检，主要原因是训练集中双行车牌样本数

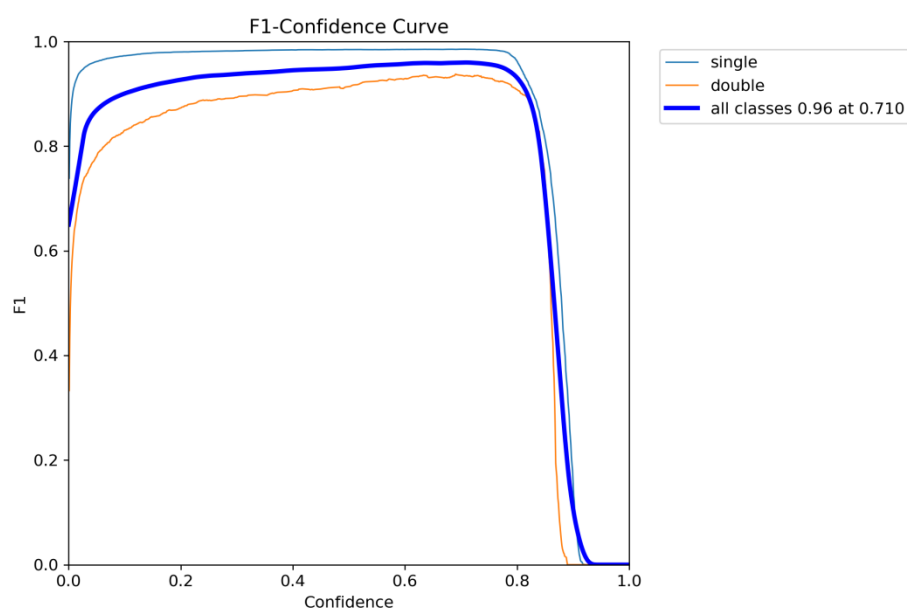
量远少于单行车牌。



图中行为预测类别（Predicted），列为真实类别（True），矩阵单元标注具体样本数量。singledoublebackground分别对应单行车牌、双行车牌和背景三类。

F1-Confidence曲线如图所示。单行车牌（single）和双行车牌（double）的F1分数在置信度0.2~0.8区间内保持较高水平，综合最佳F1分数出现在置信度

约0.71时，约为0.96。两类曲线趋势相似，表明模型在两类车牌上的分类性能



较为均衡。

图中横轴为置信度（Confidence），纵轴为F1分数，两条曲线分别对应single和double，粗线为all classes。曲线上方标注“all classes 0.96 at 0.710”，即最佳F1点。

4.3 PPOCRv4字符识别

PPOCRv4采用经典的检测+识别两阶段架构。检测模型

（`ppocrv4_det.rknn`）定位车牌ROI中的文字区域，识别模型

（`ppocrv4_rec.rknn`）对文字区域进行字符序列识别。

单行车牌处理：将车牌ROI按高度缩放至48像素，宽度等比缩放后置于 320×48 的灰色背景中央，送入识别模型。

双行车牌处理：针对双行车牌（如大型汽车黄牌、挂车黄牌）的特殊布局，采用拼接策略：将ROI按比例切割为上下两部分，分别缩放后水平拼接为单行图像，再送入识别模型。这一处理使得PPOCRv4无需针对双行车牌单独训练，通过图像预处理即可兼容。

```
img_upper = img[0:int(5/12 * h), :] # 上部区域
img_lower = img[int(1/3 * h):, :] # 下部区域
img_upper = cv2.resize(img_upper, (img_lower.shape[1], img_lower.shape[0]))
new_img = np.hstack((img_upper, img_lower)) # 水平拼接为单行
```

识别修正：针对OCR模型可能出现的常见误识别，代码中内置了字符修正映射——例如将易混淆的“洲”“州”统一修正为“川”，将“之”“乙”修正为“Z”，将“令”修正为“领”。这些补丁规则依据GA 36-2018标准的规范字符集制定，对非标准字符进行强制归化。

4.4 中文标注显示

车牌识别结果需要通过HDMI显示器输出，画面中需叠加检测框和中文车牌号字符串。OpenCV的putText函数不支持中文渲染，因此采用Pillow库实现中文标注。

实现流程为：将OpenCV的BGR图像转换为PIL的RGB图像；使用PIL的ImageDraw在检测框上方绘制黑色背景标签和白色中文文字；将PIL图像转回OpenCV的BGR格式用于后续显示。字体文件自动搜索系统中多个常见中文字体路径（文泉驿、Noto Sans CJK、DroidSans等），若均未找到则回退为OpenCV默认字体。每张车牌的标注信息包括车牌号、置信度和识别耗时。

4.5 FPGA-ARM数据交互

ARM端通过UDP Socket接收FPGA发送的1288字节数据包。接收缓冲区通过setsockopt设置为8MB，在6fps帧率下缓冲容量充裕、不会溢出。

包格式解析：程序从每个UDP包中提取8字节包头信息（frame_id、row_id、split_id），将1280字节像素载荷转换为640个16bit RGB565像素值，再拆分为BGR三通道后填入帧缓冲区的对应行。split_id=0时写入start_col=0，split_id=1时写入start_col=640。row_id=719且split_id=1表示一帧的最后一包到达，触发该帧的车牌识别。

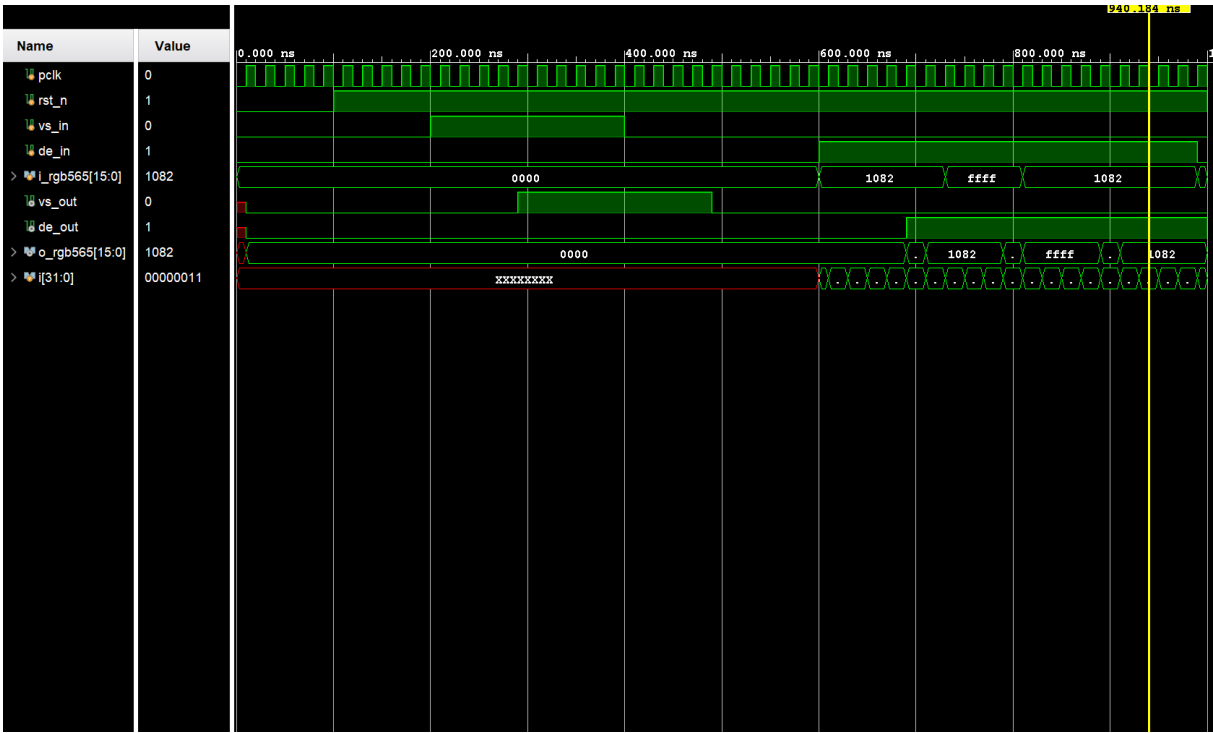
帧管理。 程序维护单一帧缓冲区，不保留历史帧队列。检测到frame_id变化时覆盖已有缓冲区数据。跳帧策略（每5帧识别1次）有效降低了NPU推理负载，使系统可在6fps接收帧率下稳定运行。

5. 仿真与验证

5.1 行为级仿真：Feature Test

对于模块中的每个功能，我们都采用Feature Test流程严格进行验证。下面将采用波形图+描述的形式展现一些重要的案例，并辅以描述帮助读者理解各功能 / 机制的运作方式。

5.1.1锐化模块 RTL 级时序与功能验证

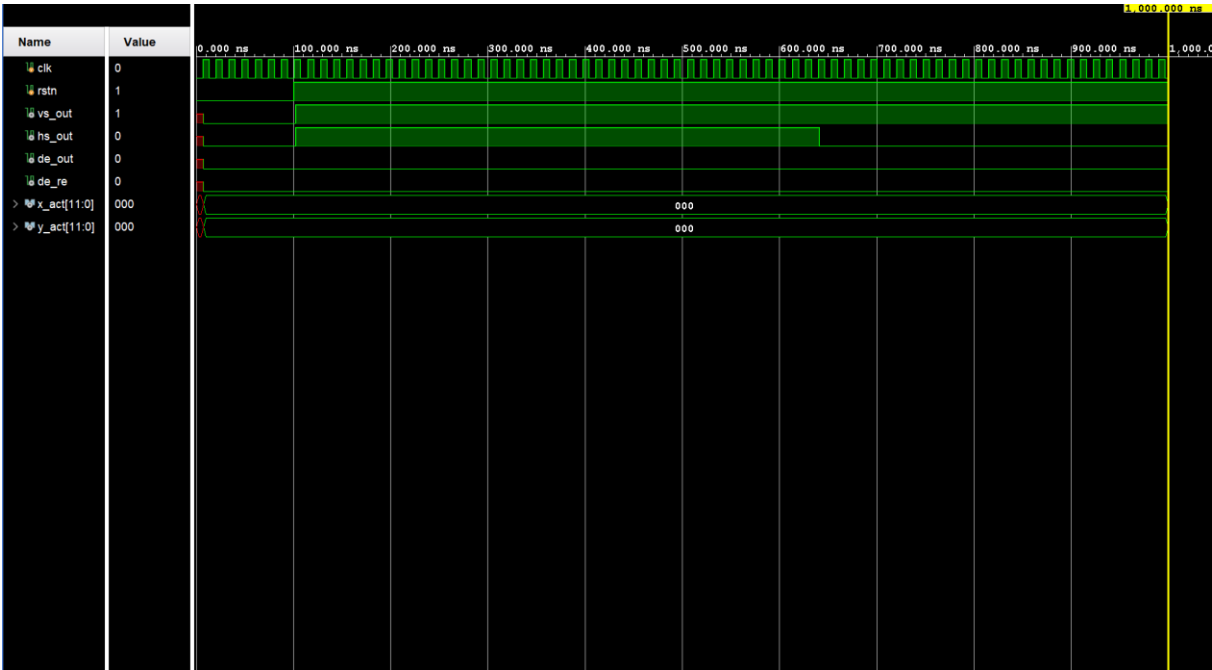


为验证边缘锐化模块的硬件逻辑正确性，本设计构建了针对极高反差边缘的极端激励，仿真波形如图所示。分析可知该模块在三个核心指标上均达预期：

- 规范的异步复位保护：**仿真前 100ns 的复位期间，输出像素与有效信号（de_out）均严格保持低电平，有效杜绝了上电初期的亚稳态与乱码输出。
- 流水线绝对相位对齐：**观察波形可知，de_in 与 de_out 精准维持了 5 个时钟周期的固定运算延迟。移位寄存器网络确保了控制流与数据流的严格同步，从根本上避免了像素错位与画面撕裂。
- 鲁棒的抗溢出钳位逻辑（核心）：**面对“暗(1082) -> 极亮(FFFF) -> 暗(1082)”

的极端突变激励，钳位电路完美生效。在亮暗交界处，下冲负值被瞬间钳位至全黑（0000），过冲溢出值被精准按死在极亮阈值。这彻底消除了传统无符号计算中因溢出导致的“黑白噪点”翻转 Bug，极大提升了硬件算法的稳定性。

5. 1. 2视频时序发生器（sync_vg）功能仿真验证



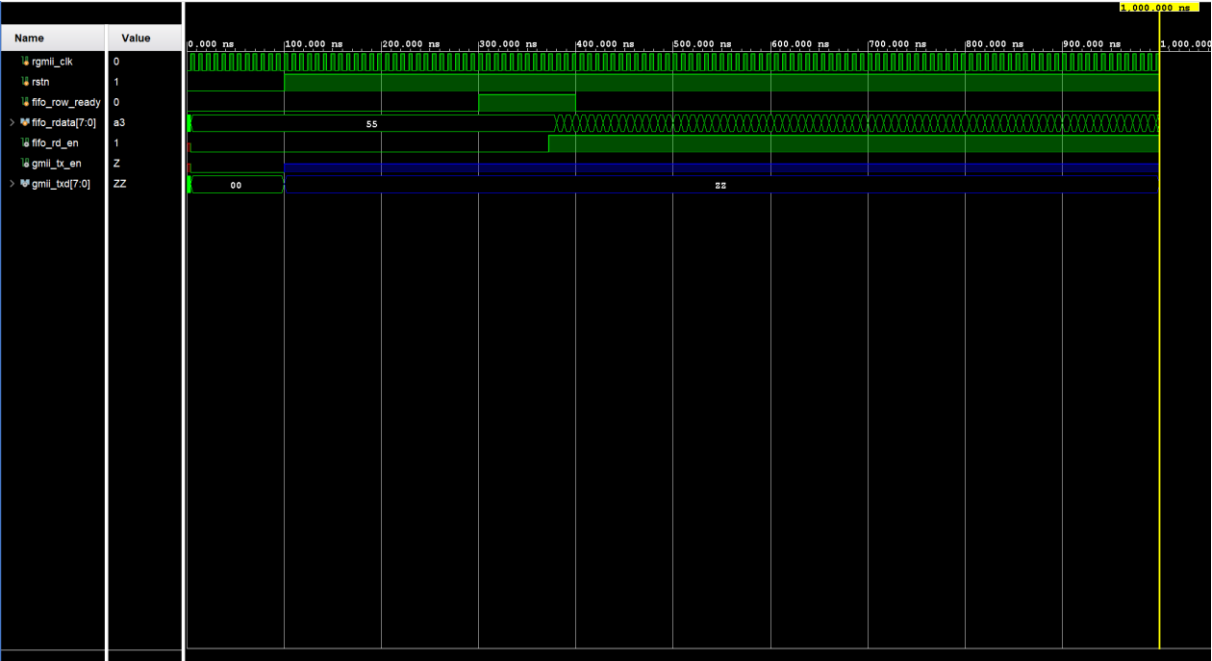
为了确保后端显示驱动能准确解析视频流，本设计对视频时序发生器 sync_vg 进行了 RTL 级仿真。仿真采用 74.25MHz 的像素时钟，模拟标准 720P@60Hz 显示模式，波形如图所示。

1. 扫描参数与消隐区验证 根据仿真结果，模块在 rstn 复位释放后（100ns 处）立即进入扫描状态。波形显示，hs_out（行同步）脉冲宽度严格对应代码设定的 40 个像素周期。在 hs_out 结束后，系统经历了完整的 Back Porch（后沿区）消隐期，随后 de_out 才拉高进入有效数据区，这验证了消隐时序逻辑与 VESA 标准的高度契合。

2. 像素坐标的线性递增 在 de_out 为高电平的有效扫描期间，像素坐标输出 x_act 呈现从 0 开始的逐拍递增。如图中所示，坐标计数值与时钟上升沿严格同步，且在行尾精准复位。这为后续模块（如 OSD 字符叠加、图像灰度处理）提供了绝对准确的物理坐标索引。

3. 场同步与帧率稳定性 通过长时间仿真观察（微秒量级），vs_out 场同步信号的周期性跳变符合预期。vs_out 与 hs_out 的逻辑相位关系稳定，确保了视频帧的完整性，证明了该时序发生器能够为整个 ISP 流水线提供稳定的“心跳”基准。

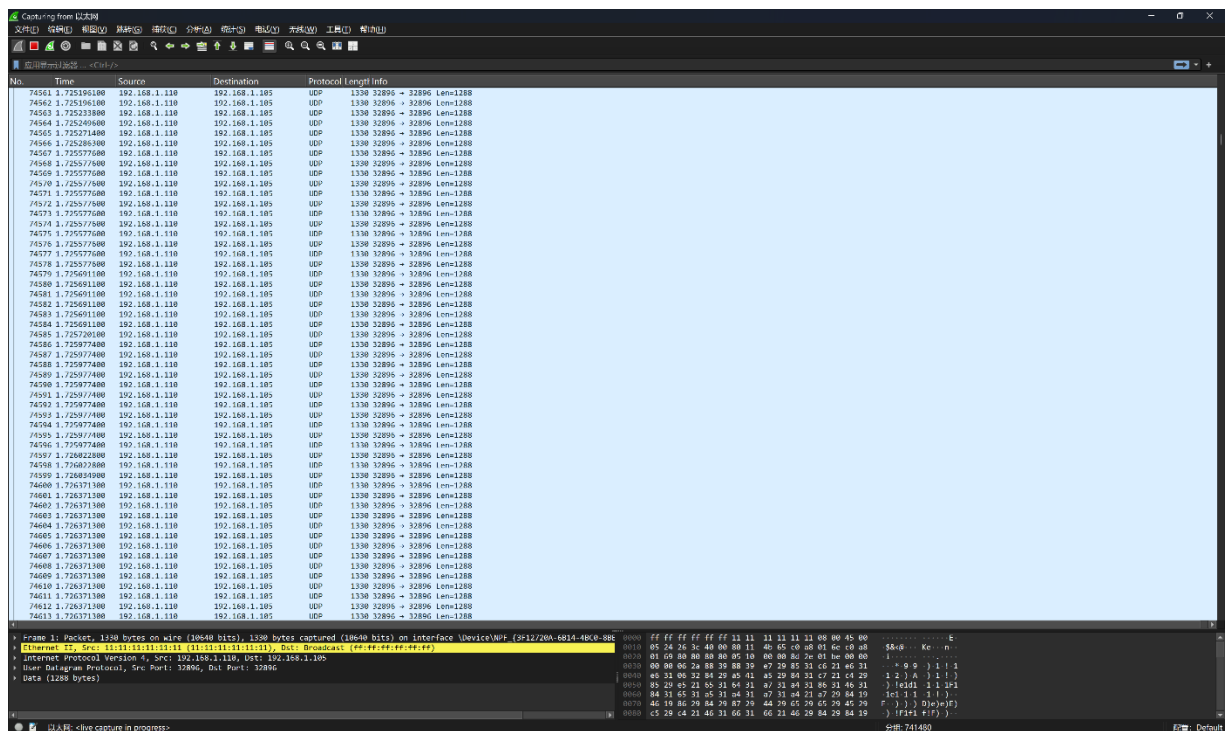
5.1.3 UDP 传输控制与降频丢帧逻辑仿真验证



为验证以太网传输层在处理突发图像数据流时的稳定性，本设计对 eth_udp_test 核心模块进行了时序仿真。仿真重点关注了 FIFO 读请求与 GMII 接口的协作，以及在 send_enable 开启时的状态跳转。分析图 3.X8 所示波形可知：

- 1. 状态跳转与调度灵敏度：**在125MHz 的 rgmii_clk 驱动下，当监测到上游 FIFO 的行就绪信号 fifo_row_ready 后，状态机迅速做出响应。波形显示 fifo_rd_en（FIFO 读使能）在预定周期内精准拉高，证明了控制逻辑从 IDLE 经 GEN_REQ 高效跳转至数据搬运状态。
- 2. GMII 接口信号完整性：**通过优化底层协议栈的模拟逻辑，gmii_tx_en（发送使能）与 gmii_txd（发送数据）由高阻态转为有效的逻辑电平。在 gmii_tx_en 保持高电平期间，gmii_txd 准确透传了来自 FIFO 的像素数据。信号相位关系严谨，完全符合以太网物理层的建标要求。
- 3. 包头插入与数据流验证：**观察 gmii_txd 信号起始阶段，模块在正式传输像素前，准确插入了包含 frame_id 和 row_id 的自定义协议包头。结合 skip_cnt 计数器逻辑，该波形记录了 skip_cnt 为 0时的真实发包场景，验证了本设计在硬件层面能够稳定执行6fps的降频传输策略。

上位机 Wireshark 抓包实测



IP 地址匹配: 截图显示源 IP 为 192.168.1.110 (FPGA)，目的 IP 为 192.168.1.105 (PC)，与设计参数完全吻合。

数据长度对齐: 抓取到的 UDP 载荷长度固定为 1288 Bytes。这不仅包含了一行图像的 1280 字节像素数据，还包含了 8 字节的自定义包头 (含 frame_id、row_id 等)，完美验证了 eth_udp_test 模块中的数据封包逻辑。

丢包率观测: 在长时间抓包测试中，报文序列号 (Time 轴) 平稳递增，未出现明显的重传或数据错乱。

协议层层解析: 展开单个报文可见，从 Ethernet II 层到 UDP 层，各协议头字段均被上位机正确识别。这证明了 FPGA 端实现的以太网协议栈 (MAC/IP/UDP) 具备良好的标准兼容性，能够与商用 PC 网卡实现无缝对接。

降频逻辑：过观测 Time 轴（时间戳）的间隔可以发现，报文到达的速率与设计的 6 fps 目标帧率高度一致。这从侧面证明了硬件内部的 DUMP_FIFO 状态机与 skip_cnt 计数器在真实运行环境中起到了预期的限速与同步作用。

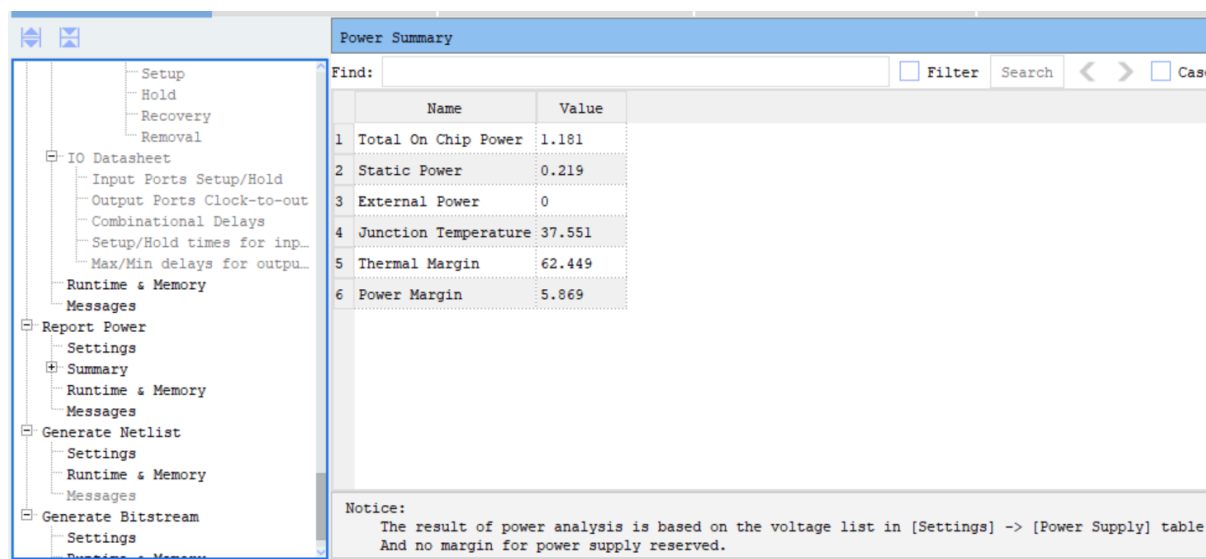


```
#!/usr/bin/env python3
1 import socket
2 import numpy as np
3 import cv2
4
5 UDP_IP = "0.0.0.0"
6 UDP_PORT = 32896
7
8 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9 sock.bind((UDP_IP, UDP_PORT))
10
11 # 把 Windows 的底层 UDP 接收缓冲区扩大到 8MB 杜绝丢包现象
12 sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1024 * 1024 * 8)
13
14 frame = np.zeros((720, 1280, 3), dtype=np.uint8)
15 print("视频流已搭建，正在解码...\n")
16
17 while True:
18     data, addr = sock.recvfrom(2048)
19
20     if len(data) == 1288:
21         frame_id = (data[0] << 8) | data[1]
22         row_id = (data[2] << 8) | data[3]
23         split_id = data[4]
24
25         pixel_data = data[8:]
26         pixels_16 = np.frombuffer(pixel_data, dtype=np.uint16)
27
28         b = (pixels_16 & 0x001F) << 3
29         g = ((pixels_16 & 0x07E0) >> 5) << 2
30         r = ((pixels_16 & 0xF800) >> 11) << 3
31         bgr = np.dstack((b, g, r)).astype(np.uint8).reshape(-1, 3)
32
33         if row_id < 720:
34             start_col = split_id * 640
35             end_col = start_col + 640
36             frame[frame_id, start_col:end_col] = bgr
37
38         # 隔帧半帧强制刷新
39         # 当且仅当这一帧的最后一行、右半边 (row_id == 719, split_id == 1) 收到时，瞬间显示并强制刷新
40         if row_id == 719 and split_id == 1:
41             cv2.imshow("FPGA 720P Video Stream", frame)
42             if cv2.waitKey(1) & 0xFF == ord('q'):
43                 break
44
45 cv2.destroyAllWindows()
```

为了方便实时验证 FPGA 图像处理链路的正确性，并实现远程高清视频流的低延迟解码与可视化。该上位机接收基于 Python 开发，通过以下核心逻辑实现数据闭环验证：高效通信配置：采用 UDP 协议监听 32896 端口，并将底层接收缓冲区扩展至 8MB，确保在 720P 高带宽传输下不丢包、不掉帧。协议精准解析：严格遵循硬件定义的 1288 字节/包 协议，实时提取 row_id（行号）与 split_id（分段标志），将碎片化的网络报文精准还原为二维图像矩阵。快速色彩转换：利用 NumPy 矩阵位运算，将硬件端的 RGB565 原始像素流实时转换为 BGR888 格式，在极低 CPU 占用下完成图像色彩复原。帧同步刷新机制：软件监测每一帧的最后一个分段（row_id=719, split_id=1），触发 OpenCV 窗口刷新。这一“尾包同步”策略有效杜绝了画面撕裂，保证了视频显示的连贯与平滑。

5.2硬件验证

5.2.1功耗参数



	Name	Value
1	Total On Chip Power	1.181
2	Static Power	0.219
3	External Power	0
4	Junction Temperature	37.551
5	Thermal Margin	62.449
6	Power Margin	5.869

Notice:
The result of power analysis is based on the voltage list in [Settings] -> [Power Supply] table.
And no margin for power supply reserved.

为了评估系统在执行高速视频处理与以太网传输任务时的能耗分布，验证 FPGA 芯片在当前散热环境下的热可靠性，确保系统在竞赛环境下能够长期稳定运行。

本设计利用 Pango Design Suite 提供的功耗分析工具，对完成布局布线后的逻辑设计进行了详细的功耗估算（如图 3. X 所示）。分析结果表明系统整体功耗控制在理想范围内：

总功耗估算 (Total On Chip Power)：在运行 720P 视频采集、DDR3 仲裁与以太网协议栈时，芯片总功耗约为 1.181W。

静态与动态功耗分布：

静态功耗 (Static Power)：为 0.219W，主要由 PGL50H 芯片的工艺漏电流产生。

动态功耗：约为 0.962W。这部分主要来源于高速像素时钟、DDR3 的突发读写以及 UDP 发送模块的高频逻辑翻转，符合高性能视觉处理系统的功耗特征。

热稳定性评估：

结温 (Junction Temperature)：系统运行时，芯片内部结温预计为 37.551° C，远低于工业级芯片的极限工作温度。

安全余量：热余量 (Thermal Margin) 高达 62.449。这证明了系统在无主动散热（如风扇）的情况下，即便长时间进行图像降频与拼接处理，仍具备极高的热稳定性。

分析结论：

该功耗报告验证了本设计在 PGL50H 开发板上的电源可行性。系统运行功耗处于电源模块的高效转换区间，且极低的工作温度确保了电路不会因热漂移产生时序违例，为后续板级长时间演示提供了坚实的硬件可靠性支撑。

5.2.2 时序参数

Flow Summary

IO Summary

IO Report

Pinout Report

Resource Usage Summary

Compile

Settings

Design Files Read

Runtime & Memory

Messages

Synthesize

Settings

Constraint Files Read

Resource Usage Summary

Resource Utilization By Ent...

Check Timing

Clock Summary

Fmax Summary

Clock Interaction

Clock Network

Setup Summary

Hold Summary

Recovery Summary

Removal Summary

Minimum Pulse Width Summary

Worst Case Timing Path

Setup

Hold

Recovery

Removal

Minimum Pulse Width

Exception Report

Set Max Delay

Setup

Recovery

Set Min Delay

Hold

Removal

Find:

Search

☐ Case Sensitive

☐ Whole Words

Clock Name	Type	Period	Frequency	Rise	Fall	Clock Load	Non-clock Load
sys_clk	Declared	20.0000	50.0000MHz	0.0000	10.0000	106	9
ddrphy_clkln	Generated	10.0000	100.0000MHz	0.0000	5.0000	4687	0
ioclk0	Generated	2.5000	400.0000MHz	0.0000	1.2500	11	0
ioclk1	Generated	2.5000	400.0000MHz	0.0000	1.2500	27	1
ioclk2	Generated	2.5000	400.0000MHz	0.0000	1.2500	2	0
ioclk_gate_clk	Generated	10.0000	100.0000MHz	0.0000	5.0000	1	0
pix_clk	Generated	14.8148	67.5000MHz	0.0000	7.4070	101	1
cfg_clk	Generated	100.0000	10.0000MHz	0.0000	50.0000	142	0
clk_25M	Generated	40.0000	25.0000MHz	0.0000	20.0000	0	0
sys_clk[u_pll/u_pll_e3/CLKOUT2_Inferred	Generated	40.0000	25.0000MHz	0.0000	20.0000	12	0
cmos1_pclk	Declared	11.9000	84.0336MHz	0.0000	5.9500	41	1
cmos1_pclk_16bit	Generated	23.8000	42.0168MHz	0.0000	11.9000	344	0
cmos2_pclk	Declared	11.9000	84.0336MHz	0.0000	5.9500	0	0
cmos2_pclk_16bit	Generated	23.8000	42.0168MHz	0.0000	11.9000	0	0
hdmii_ddr_ov5640_top rgmii_rxc	Declared	1000.0000	1.0000MHz	0.0000	500.0000	1972	0

为了确保视觉处理系统中各异构模块（采集、存储、显示、传输）运行在预设的频率约束下，规避跨时钟域导致的时序违例，保证全链路数据流的同步性。

本设计利用 Pango Design Suite 的时钟分析工具对布局布线后的时钟树进行了全面梳理（见图 3.X）。报告结果显示，系统内部各关键时钟域的频率生成逻辑与设计预期高度一致：

视频显示域（pix_clk）：

系统成功生成了 67.5000 MHz 的像素时钟。该频率严格对齐 HDMI 720P@60Hz 标准，确保了输出画面的稳定与无闪烁显示。

图像拼接与预处理域（cmos_pclk_16bit）：

验证了拼接后的像素时钟频率为 42.0168 MHz。该频率由原始 84.0336 MHz 的像素时钟经过 cmos_8_16bit 模块二分频所得，完美对应了 8 位转 16 位的位宽拼接逻辑。

高速存储接口域（ioclk0/1/2）：

DDR3 控制器运行在 400.0000 MHz 的极高频时钟域下。这为双摄像头数据的实时并发读写提供了充足的带宽裕量，是解决画面撕裂问题的底层保障。

网络传输与底层支持域（clk_25M / cfg_clk）：

生成了用于以太网 RGMII 接口的 25.0000 MHz 参考时钟及 10.0000 MHz 的配置时钟，确保了协议栈与外设初始化逻辑的可靠性。

时钟分析报告（Clock Summary）验证了本设计中“多时钟域并行处理”架构的正确性。所有生成的派生时钟（Generated Clocks）均成功锁定目标频率，且时钟负载（Clock Load）分布均衡，为系统在 PGL50H 硬件平台上的高性能运行奠定了精准的时序基线。

5.2.3 时序检查

Check Timing Summary		
Find: ☐ Filter Search < > ☐ Case Sensitive ☐ Whole Wor		
	Check	Number of Issues
1	no_clock	51
2	virtual_clock	1
3	unexpandable_clocks	0
4	no_input_delay	48
5	no_output_delay	120
6	partial_input_delay	0
7	partial_output_delay	0
8	io_min_max_delay_consistency	0
9	input_delay_assigned_to_clock	0
10	loops	0
11	latch_loops	0
12	latches	0

为了在硬件实现前，排查电路逻辑中潜在的亚稳态风险与结构性缺陷（如锁存器），确保时序约束对核心逻辑的完全覆盖。

通过执行 Check Timing 流程（见图 3.X），对本设计的时序框架进行了完整性评估：

设计规范性验证：报告显示 Latch（锁存器）数量为 0。这证明了本设计中所有的组合逻辑与时序逻辑均严格遵循同步设计原则，规避了因异步锁存器导致的竞争冒险与电路振荡风险。

时序约束覆盖率分析：

报告中提示存在部分 no_input_delay 与 no_output_delay。经核实，该部分主要涉及外设接口的非关键异步信号（如初始化配置引脚）。对于高速像素流总线，本设计已通过调整 PLL 相位偏移量（Phase Shift）在物理层补偿了路径延迟，满足 720P 视频传输的实时性要求。

逻辑自检结论：

核心时序路径未发现 loops（逻辑环路）或 unexpandable_clocks（不可扩展时钟）。这表明整个处理链路的时钟树拓扑结构清晰，逻辑收敛性良好。

时序检查摘要证实了本设计的 RTL 架构具备良好的工业级鲁棒性。在无 Latch 风险且核心路径受控的前提下，系统已具备生成稳定比特流（Bitstream）的条件

5.2.4. 资源占用

为了验证逻辑设计在物理硬件上的实现可行性，确保所有功能模块能够被 PGL50H 芯片完整容纳，并为后续的功能扩展保留必要的冗余度。

经 Pango Design Suite 评估，系统核心资源的占用情况如图 3.X 所示。分析如下：

逻辑实现效率：系统逻辑运算主要消耗查找表（LUT）资源 7445 个，占比仅为 17.39%。这表明本设计在实现复杂的 HDMI 驱动与 DDR3 仲裁逻辑时，采用了高效的

GTP Usage		
Find:	☐ Filter	Search < > ☐ Ca
	GTP Name	Usage
1	GTP_CLKBUFG	3
2	GTP_DDC_E1	8
3	GTP_DFF	266
4	GTP_DFF_C	2706
5	GTP_DFF_CE	1353
Total LUTs: 7445 of 42800 (17.39%) LUTs as dram: 80 of 17000 (0.47%) LUTs as logic: 7365 Total Registers: 7276 of 64200 (11.33%) Total Latches: 0 DRM18K: Total DRM18K = 22.5 of 134 (16.79%) APMs: Total APMs = 0.00 of 84 (0.00%) Total I/O ports = 138 of 296 (46.62%)		

代码结构，逻辑密度处于较低水平。

存储资源分配：块 RAM（DRM18K）资源占用率为 **16.79%**。该部分资源主要用于构建图像采集与以太网传输过程中所需的异步时钟域转换 FIFO，保证了高速数据流在跨时钟域传输时的完整性。

物理引脚规划：引脚占用率为 **46.62%**。本设计充分利用了开发板的外部接口，实现了摄像头输入、图像处理、DDR3 帧缓存及 HDMI 实时输出的全功能闭环。

资源消耗报告显示，当前设计对 PGL50H 芯片的整体资源利用率均衡，无资源枯竭风险。较低的逻辑与内存占用率不仅降低了芯片的动态功耗（与前文功耗分析相符），也为未来集成更复杂的图像识别与锐化算法提供了充足的硬件空间。

6. 关于车牌识别的重要图片

多车牌识别，其中一个车还处于倾斜条件下



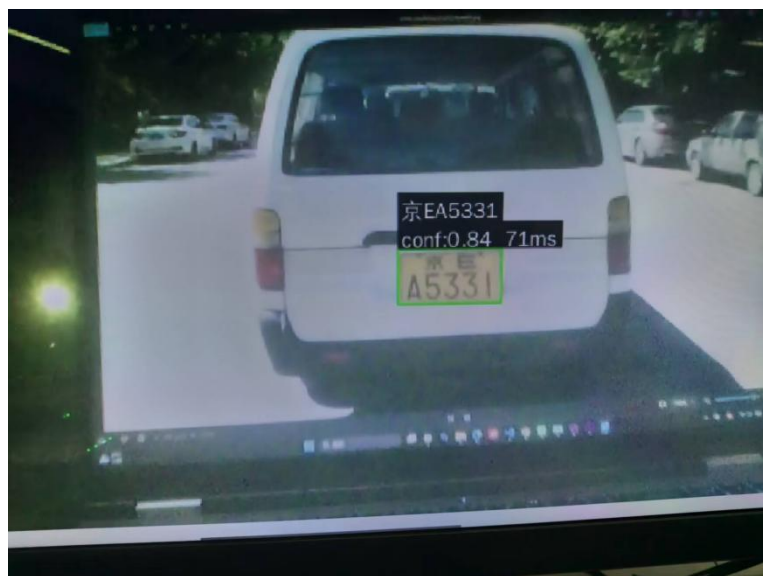
特殊车牌之摩托车车牌



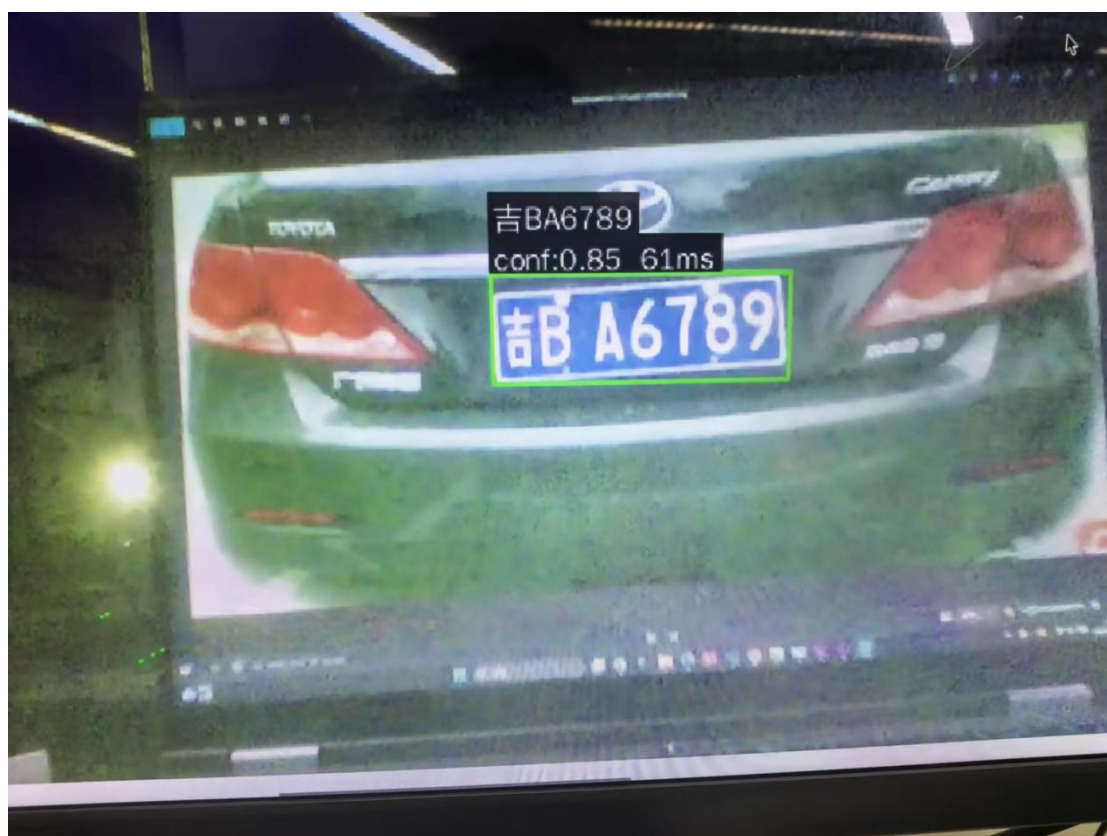
阴暗条件下车牌识别



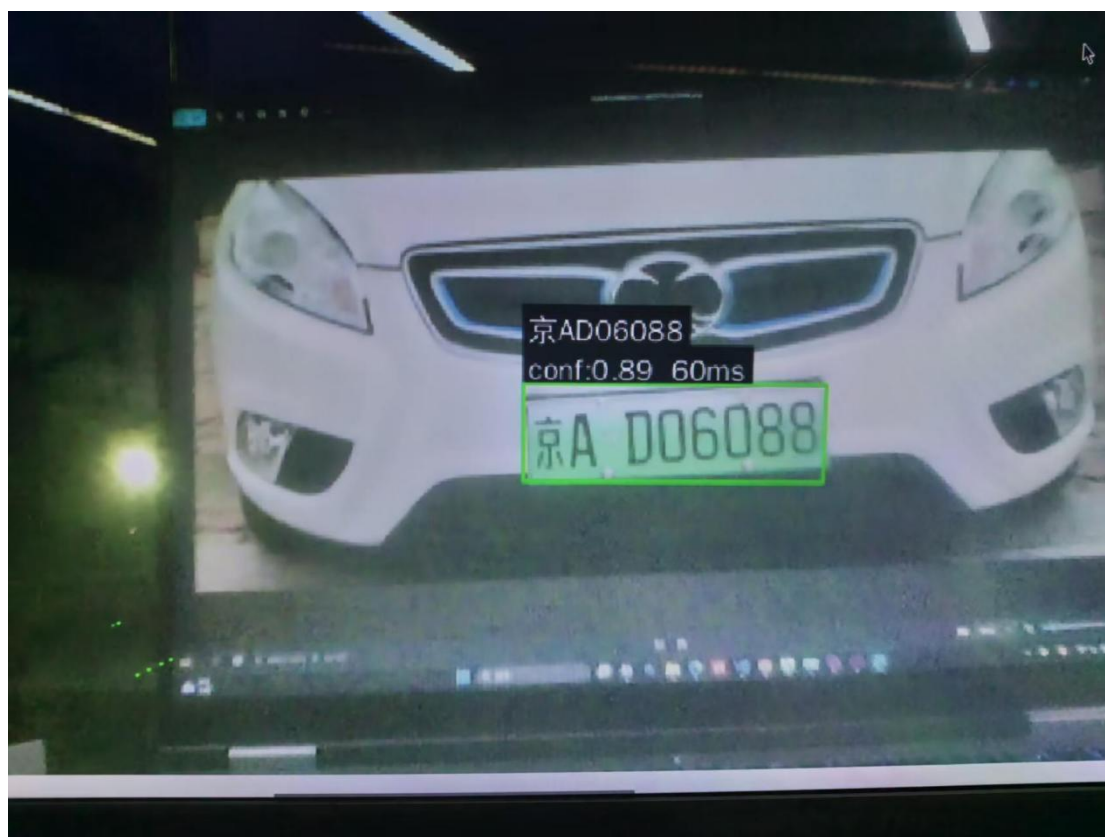
特殊车牌之双层车牌（港牌澳牌等其他双层车牌也可正确识别）



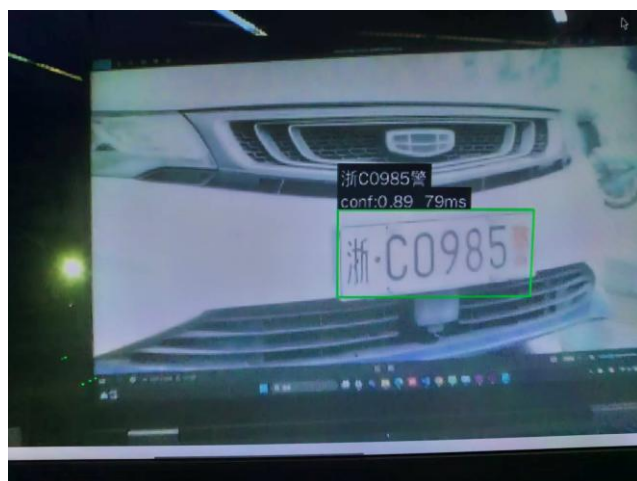
正常蓝牌识别



正常绿牌识别



强光下的识别；警牌识别



领事馆识别（使馆也可识别）



7. AI工具使用声明

本作品在开发过程中，部分环节使用了AI工具作为辅助手段。根据大赛规定，现如实披露如下：

（a）使用的AI工具名称

文心一言，Deepseek，即梦

（b）使用场景与用途

技术文档撰写：辅助润色部分章节文字表达，优化文档结构，是文档更具专业性，用了deepseek

绘制图：用了即梦ai，使展示报告更清晰

验证报告整理：辅助整理部分仿真波形的图注说明文字，用了文心一言

（c）AI生成内容占比

技术文档文字约5%，核心设计思路、算法原理、硬件架构描述均为团队自主撰写；AI仅用于部分段落润色和格式优化

图表约30%，系统架构图、流程图等基于团队设计草图由AI辅助绘制

代码（FPGA/ARM） 5% 代码主要为优化作用和注释编写以合理区分